Online Medicines Ordering Database

To design and implement a secure, efficient, and scalable database system for an online medicine ordering platform that ensures accurate data management, quick retrieval, and seamless integration with various system components while maintaining data integrity and security.

Objectives:

- **Data Storage & Organization** – Structure the database to store essential information, including Customer details, medicines, orders, payments records.
- **Efficient Data Retrieval** – Optimize queries for fast and accurate retrieval of medicines, customer information, and order status.
- **Data Integrity & Consistency** – Implement constraints and normalization techniques to maintain data accuracy and prevent duplication.
- **Inventory Management** – Maintain real-time stock levels of medicines and prevent over-ordering.
- **Order Tracking & History** – Keep records of past and ongoing orders, including timestamps, payment status, and delivery updates.
- **Payment Processing Records** – Securely store payment transaction details while ensuring compliance with financial security standards.
- **Scalability & Performance Optimization** – Design the database to handle increasing customers, orders, and transactions without performance degradation.
- **Data Backup & Recovery** – Implement regular backups and recovery mechanisms to prevent data loss due to failures

This database will serve as the backbone of the online medicine ordering system, ensuring reliability, security, and seamless operation.

Database Overview

Data :

Data refers to any piece of information. It can be numbers, words, images, sounds, or any other information that a computer can store and process. Data can be raw or processed.

Database:

A database is a structured collection of data that is organized in a way to facilitates efficient storage, retrieval, and manipulation of information. It acts as a centralized and organized repository where data can be stored, managed, and accessed by various applications or users.

Database Management System :

A Database Management System (DBMS) is a software system that enables users to define, create, maintain, and manipulate databases. It provides an interface between the database and the users or applications, allowing them to access and manage data efficiently.

Types of Databases :-

There are different types of databases bases on how we organize the data :

1) Relational Databases (SQL Database):

Relational databases(RDBMS) organize data into tables, where each table has rows and columns. These databases use structured query language (SQL) for defining and manipulating data.

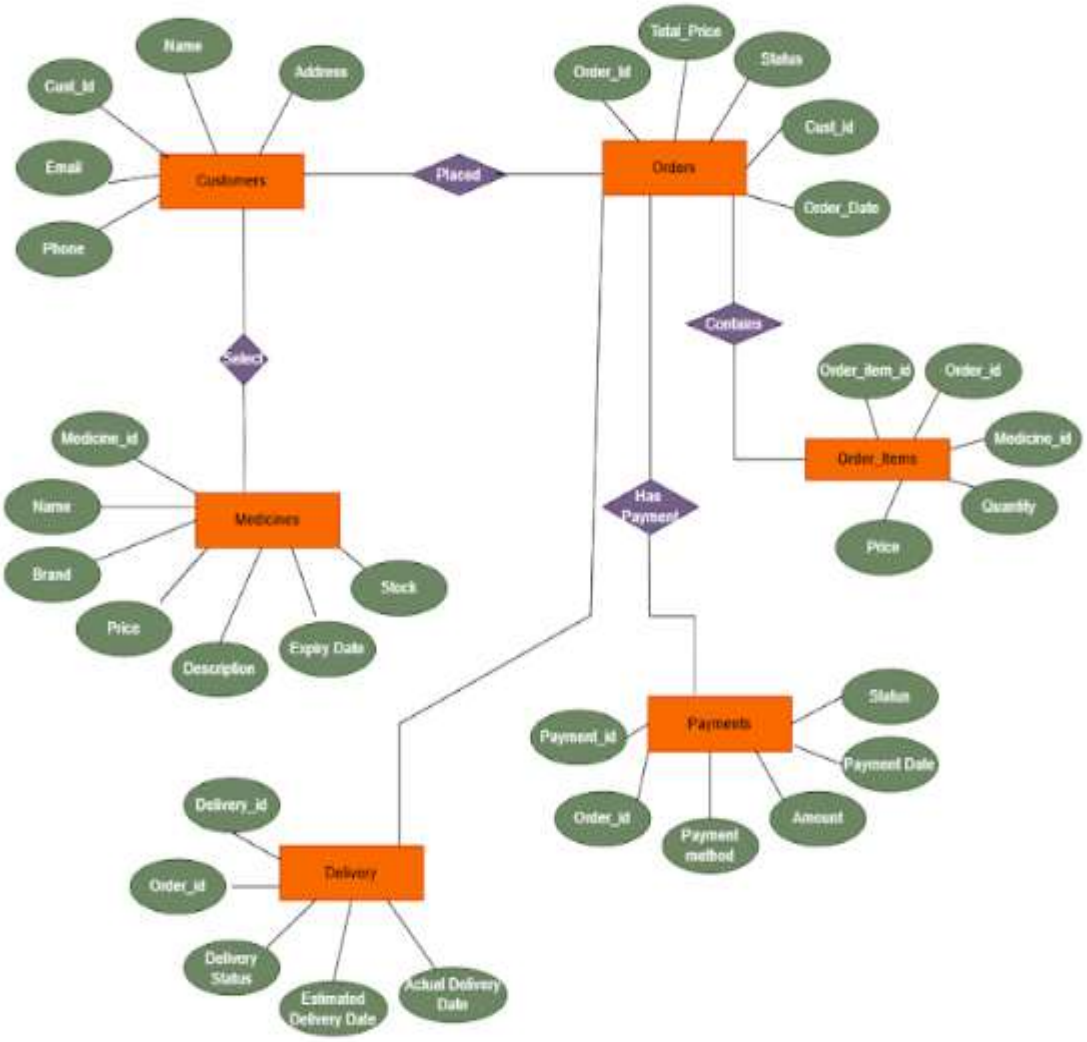
MySql, PostgreSQL, SQL Server, MariaD this are refered as relational database.

2) Non-Relational Database (NoSQL Database) :

These databases are designed to handle large volumes of unstructured, semi-structured, or structured data. NoSQL provide flexible schema designs and often offer horizontal scalability.

Document , Graph, key value, Document. This are commonly refered as non- relational databases.

ER- Diagram



The ER diagram shows how a Medicines Ordering database is organized. Here are the main parts:

1. Customers: This section stores details about each customer, like their contact information.

2. Medicines: This section stores details about each Medicines, including its name, brands, price, quantity.

3. Orders: Orders track Which Medicines customer buys. Each order links to the

customers, the Medicines. When the user places an order, a new entry is created in the Orders table.

4. Order_Item: One order can have multiple medicines, each medicine in the order is stored separately in the Order_Items table.

5. Payment: Payments are connected to each order and contain details like the payment amount and method.

6. Delivery: This section tracks the delivery of orders, including the delivery person and the delivery status.

Normalization

Normalization is a database design technique used to organize data into tables in such a way that reduces redundancy and dependency of data.

It ensures data integrity by eliminating insertion, deletion, and update anomalies.

Goals of Normalization:

- Eliminate redundant data: Reduces storage space and improves data consistency.
- Minimize data anomalies: Ensures that modifications to the database do not introduce inconsistencies.
- Improve data integrity: Ensures that data is accurate and reliable for querying and reporting.

Why is this required?

Without Normalization in SQL, we may face many issues such as

- Insertion anomaly: It occurs when we cannot insert data to the table without the presence of another attribute
- Update anomaly: It is a data inconsistency that results from data redundancy and a partial update of data.
- Deletion Anomaly: It occurs when certain attributes are lost because of the deletion of other attributes.

First Normal Form (1NF)

This normalization level ensures that each column in your data contains only atomic values. Atomic values in this context means that each entry in a column is indivisible.

It is like saying that each cell in a spreadsheet should hold just one piece of information.

1NF ensures atomicity of data, with each column cell containing only a single value and each column having unique names.

Second Normal Form (2NF)

Eliminates partial dependencies by ensuring that non-key attributes depend only on the primary key.

What this means, in essence, is that there should be a direct relationship between each column and the primary key, and not between other columns.

Third Normal Form (3NF)

Removes transitive dependencies by ensuring that non-key attributes depend only on the primary key. This level of normalization builds on 2NF.

Example : Normalized the Unnormalized data of given table..

Order Id	Quantity	Item price	Total cost	Order date	status	Medicine Id	Medicine name	Medicine price	Customer name	Contact	Address	Payment method
101	3	30	90	04-11-24	Deliverd	201	Paracetamol	20	Saya S	9876543219	Pune	Debit card
102	5	60	300	27-02-25	Deliverd	202	Cough Syrup	50	Nikita G	7878676756	Vashi	Cash On Delivery
103	4,2	30-20		28-02-25	Shipped	202,203	Cough Syrup, D	25-15	Parth F	7867564534	Thane	UPI

Problems in the table:

1. Multi-valued attributes:

- Quantity, Item Price, Medicine Id, Medicine Name, and Medicine Price contain multiple values in a single row (Order 103).

2. Data redundancy:

- Customer details (Customer Name, Contact, Address) are repeated for each order.
- Medicine details (Medicine Id, Medicine Name, Medicine Price) are repeated across multiple orders.

3. Partial and Transitive Dependencies:

- Customer Name, Contact, and Address depend on Order Number, not individual medicines.
- Medicine Price depends on Medicine Id, not Order Number

Here, unnormalized data is normalized till 3NF i.e., Third Normal Form.

First Normal Form (1NF) :

To achieve 1NF, we need to eliminate multi-valued attributes by converting rows with multiple values into separate rows.

Order Id	Quantity	Item Price	Total Cost	Order Date	Status	Medicine Id	Medicine Name	Medicine Price	Customer Name	Contact	Address	Payment Method
101	3	30	90	04-11-24	Delivered	201	Paracetamol	20	Saya S	9876543219	Pune	Debit Card
102	5	60	300	27-02-25	Delivered	202	Cough Syrup	50	Nikita G	7878676756	Vashi	Cash On Delivery
103	4	30	120	26-02-25	Shipped	202	Cough Syrup	50	Parth F	7867564534	Thane	UPI
103	2	20	40	26-02-25	Shipped	203	Dolo	15	Parth F	7867564534	Thane	UPI

- 1NF Achieved: Each cell contains atomic values, and multi-valued attributes are removed.

Second Normal Form (2 NF) :

To achieve 2NF, we remove partial dependencies by dividing data into separate tables.

Order Table :

Order Id	Order Date	Status	Customer ID	Payment Method
101	04-11-24	Delivered	C001	Debit Card
102	27-02-25	Delivered	C002	Cash On Delivery
103	26-02-25	Shipped	C003	UPI

Customer table :

Customer ID	Customer Name	Contact	Address
C001	Saya S	9876543219	Pune
C002	Nikita G	7878676756	Vashi
C003	Parth F	7867564534	Thane

Order-Items table :

Order Id	Medicine Id	Quantity	Item Price	Total Cost
101	201	3	30	90
102	202	5	60	300
103	202	4	30	120
103	203	2	20	40

Medicines table :

Medicine Id	Medicine Name	Medicine Price
201	Paracetamol	20
202	Cough Syrup	50
203	Dolo	15

- 2NF Achieved: No partial dependencies exist.

Third Normal Form (3 NF) :

To achieve 3NF, we need to remove transitive dependencies.

Order Id	Payment Method
101	Debit Card
102	Cash On Delivery
103	UPI

Now, Payment Method is stored separately, and Orders table only keeps Payment ID.

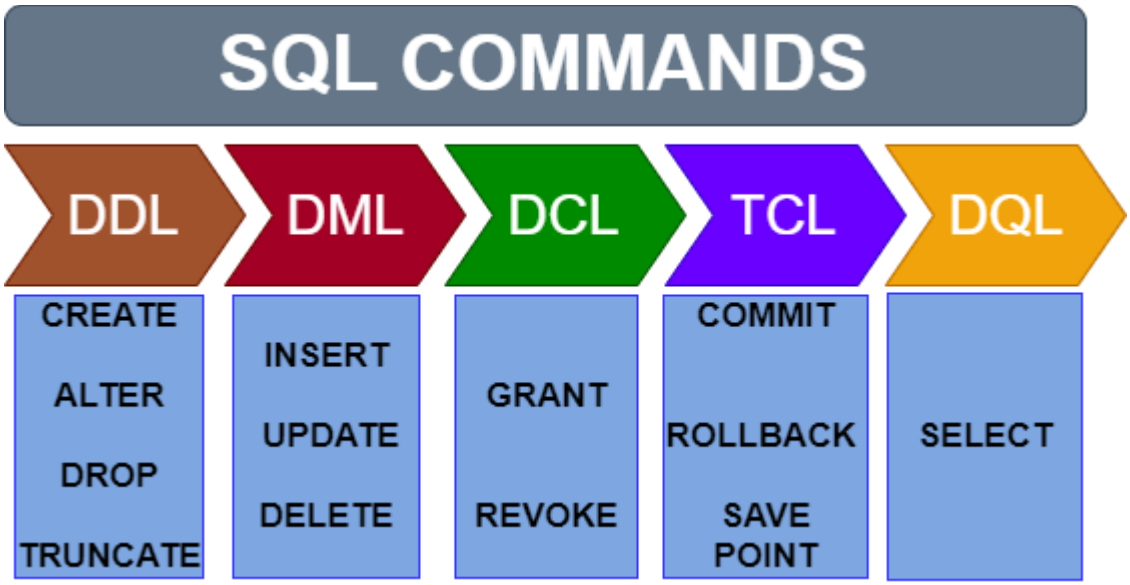
- 3NF Achieved: No transitive dependencies; all attributes depend only on the primary key.

SQL - Structure Query Language

SQL stands for Structured Query Language. It is a standardized programming language used to manage and manipulate relational databases.

It enables users to perform a variety of tasks such as querying data, creating and modifying database structures, and managing access permissions.

SQL is widely used across various relational database management systems such as MySQL, PostgreSQL, Oracle, and SQL Server.



Types of SQL Commands:

1. Data Definition Language (DDL):

- Purpose: Used for defining and managing all database objects such as tables, indexes, and schemas.
- Key Commands:
  - **CREATE**: Creates a new table or database.
  - **ALTER**: Modifies an existing database object.
  - **DROP**: Deletes a table or database.
  - **TRUNCATE** : Delete all records of the table.

2. Data Manipulation Language (DML):

- Purpose: Used for managing data within schema objects.
- Key Commands:
  - **INSERT**: Adds new records to a table.
  - **UPDATE**: Modifies existing records in a table.
  - **DELETE**: Removes records from a table.

3. Data Control Language (DCL):

- Purpose: Used for controlling access to data in the database.
- Key Commands:
  - **GRANT**: Gives a user permission to access specific database objects.
  - **REVOKE**: Removes permissions granted to a user.

4. Transaction Control Language (TCL):

- Purpose: Used to manage transactions in a database.
- Key Commands:
  - **COMMIT**: Saves all changes made during the current transaction.
  - **ROLLBACK**: Undoes changes made during the current transaction.

5.Data Query Language (DQL):

- Purpose: Used for querying and retrieving data from a database.
- Key Command:
  - **SELECT**: Retrieves data from one or more tables.

Summary

- DDL: Define structure (CREATE, ALTER, DROP)
- DML: Manipulate data (INSERT, UPDATE, DELETE)
- DCL: Control access (GRANT, REVOKE)
- TCL: Manage transactions (COMMIT, ROLLBACK)
- DQL: Retrieve data (SELECT)

Database Overview

DataBase Name : OnlineMedicinesOrderingSystem

Tables : 6

```
CREATE TABLE Customers (
    Cust_id INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) Not Null,
    Email VARCHAR(100) UNIQUE,
    Phone VARCHAR(15) UNIQUE,
    Address TEXT Not Null,
    Created_at DateTime Not Null
);
```

Cust_id	Name	Email	Phone	Address	Created_at
100	Saya Suryavanshi	saya725@gmail.com	9876543210	12, MG Road, Mumbai	2024-08-12 10:00:00
101	Nikita Giri	niki456@gmail.com	8765432109	45, Connaught Place, Delhi	2024-09-26 11:30:00
102	Parth Shirke	parth89@gmail.com	7654321098	67, Park Street, Kolkata	2024-10-15 08:45:00
103	Chirag Kamble	chirag777@gmail.com	9543210987	23, Banjara Hills, Hyderabad	2024-11-03 09:30:00
104	Ubaid Fazlani	ubaid92@gmail.com	8432109876	78, MG Road, Bangalore	2024-12-05 10:00:00
105	Dhiraj Anand	dhiraj324@gmail.com	9321098765	90, Anna Salai, Chennai	2025-01-02 11:00:00

```
CREATE TABLE Medicines (
    Medicine_id INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100)  Unique,
    Brand VARCHAR(100) Not Null,
    Price DECIMAL(10,2) Not Null CHECK Price >0,
    Stock INT Not Null,
```

```
    Description TEXT,
    ExpiryDate Date,
    Created_at DateTime Not Null
);
```

Medicine ID	Name	Brand	Price	Stock	Description	Expiry Date	Created At
200	Paracetamol 500mg	Calpol	19.00	150	Pain reliever & fever reducer	2025-06-30	2024-02-25 14:00:00
201	Amoxicillin 500mg	Mox	85.00	100	Antibiotic for bacterial infections	2026-02-15	2024-02-26 09:15:00
202	Ibuprofen 400mg	Brufen	32.50	120	Anti-inflammatory painkiller	2026-05-24	2024-03-27 12:30:00
203	Aspirin 75mg	Ecosprin	25.00	90	Blood thinner & heart attack prevention	2027-01-12	2024-04-28 08:45:00
204	Cough Syrup	Benadryl	110.00	80	Used for treating cough & throat relief	2025-07-03	2024-05-01 10:15:00
205	Vitamin C 500mg	Celin	50.00	70	Immunity booster	2027-07-28	2024-05-01 10:30:00
206	Metformin 500mg	Glycomet	58.00	65	Diabetes control medicine	2028-01-21	2024-06-02 08:00:00

```
CREATE TABLE Orders (
    Order_id INT PRIMARY KEY AUTO_INCREMENT,
    Total_price DECIMAL(10,2) Check Total_price >0 ,
    Status ENUM('Pending', 'Shipped', 'Delivered', 'Cancelled'),
    Order_date DateTime Not Null,
    Cust_id INT,
    FOREIGN KEY (Cust_id) REFERENCES Customers(Cust_id)
);
```

Order_id	Total_price	Status	Order_date	Cust_id
300	154.00	Pending	2025-01-02 10:30:00	101
301	19.00	Shipped	2025-01-25 15:00:00	102
302	110.00	Delivered	2025-02-11 09:45:00	103
303	143.00	Delivered	2025-02-16 12:00:00	104
304	182.50	Pending	2025-03-01 14:20:00	105

```
CREATE TABLE Order_Items (
    Order_item_id INT PRIMARY KEY AUTO_INCREMENT,
    Order_id INT,
    Medicine_id INT,
    Quantity INT,
    Price DECIMAL(10,2) Not Null Price>0,
    FOREIGN KEY (Order_id) REFERENCES Orders(Order_id),
    FOREIGN KEY (Medicine_id) REFERENCES Medicines(Medicine_id)
);
```

Order_item_id	Order_id	Medicine_id	Quantity	Price
1	301	201	2	38.00
2	301	202	1	85.00
3	301	203	1	32.50
4	302	201	1	19.00
5	303	205	1	110.00
6	304	204	2	50.00
7	304	201	1	20.00

```
CREATE TABLE Payments (
    Payment_id INT AUTO_INCREMENT PRIMARY KEY,
    Order_id INT,
    Payment_method ENUM('Credit Card', 'Debit Card', 'UPI', 'Cash on Delivery'),
    Amount DECIMAL(10,2) Not Null Check Amount >0,
```

```
    Payment_status ENUM('Paid', 'Pending', 'Failed'),
    Payment_date DateTime Not Null,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)
);
```

Payment_id	Order_id	Payment_method	Amount	Payment_status	Payment_date
1	301	Credit Card	154.00	Paid	2025-01-07 10:35:00
2	302	UPI	19.00	Paid	2025-02-03 15:10:00
3	303	Debit Card	110.00	Paid	2025-02-10 09:50:00
4	304	Cash on Delivery	143.00	Pending	2025-03-02 12:10:00

```
CREATE TABLE Delivery (
    DeliveryID INT PRIMARY KEY AUTO_INCREMENT,
    OrderID INT,
    DeliveryStatus  ENUM('Out for Delivery', 'Delivered', 'Returned') ,
    EstimatedDeliveryDate DATE NOT NULL,
    ActualDeliveryDate DATE,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)
);
```

DeliveryID	OrderID	DeliveryStatus	EstimatedDeliveryDate	ActualDeliveryDate
1	302	Delivered	2025-02-15	2025-02-15
2	303	Delivered	2025-02-21	2025-02-21
3	304	Out for Delivery	2025-03-08	NULL

Constraints :

SQL constraints are used to specify rules for the data in a table.

Constraints are used to limit the type of data that can go into a table. This ensures the accuracy and reliability of the data in the table. If there is any violation between the constraint and the data action, the action is aborted.

Constraints can be column level or table level. Column level constraints apply to a column, and table level constraints apply to the whole table.

The following constraints are commonly used in SQL:

1. Not Null Constraint :

The NOT NULL constraint ensures that a column cannot have a NULL value. It enforces that every row must have a valid (non-null) value in that column.

=> Write a query to ensure that empName column cannot be NULL in Employee table.

```
create table Employee(empId int primary key,
empName varchar(50) not null,
contact bigint not null
);
```

If We try to insert a NULL value into a NOT NULL column empName like

--> insert into Employee values(1, null, 9856473825);

It will result in error as :

```
ERROR: Column 'empName' cannot be NULL
```

2. Unique Constraint :

The UNIQUE constraint ensures that all values in a column or a set of columns are different from one another.

=> Write a query to ensure that the email column in the employee table contains only unique values.

```
create table Employee(empId int primary key,
empName varchar(50) not null,
contact bigint not null,
email varchar(100) Unique
);
```

Suppose we insert one record as :

-- > insert into Employee values(1, Saya, 9856473825, 'saya3475@gmail.com');

```
+-------+---------+------------+--------------------+
| empId | empName | contact    | email              |
+-------+---------+------------+--------------------+
|     1 | Saya    | 9856473825 | saya3475@gmail.com |
+-------+---------+------------+--------------------+
```

Now, If We try to insert a same email id which is 'saya3475@gmail.com' for another record like :

-- > insert into Employee values(2, 'Nikita', 8256473856, 'saya3475@gmail.com');

It will result in error as :

```
ERROR 1062 (23000):  Duplicate entry 'saya3475@gmail.com' for key  'employee.email'
```

3. Check Constraint :

The CHECK constraint is used to limit the value range that can be placed in a column.

If we define a CHECK constraint on a column it will allow only certain values for this column.

=> Write a query to ensure that the salary column does not accept negative values.

```
create table Employee(empId int primary key,
empName varchar(50) not null,
contact bigint not null,
email varchar(100) Unique,
salary decimal(7,2) check (salary >= 0 ) not null
);
```

If We try to insert negative value ' -35000 ' in salary column like :

-- > insert into Employee values(3, 'Parth, 9256253867, parth777@gmail.com', '-35000.00');

It will result an error as :

```
ERROR 3819 (HY000): Check constraint 'employee_chk_1' is violated.
```

4. Default Constraint :

The DEFAULT constraint is used to set a default value for a column.

The default value will be added to all new records, if no other value is specified.

=> Write a query to ensure that the location column of employee has a default value as Mumbai.

```
create table Employee(empId int primary key,
empName varchar(50) not null,
contact bigint not null,
email varchar(100) Unique,
salary decimal(7,2) check (salary >= 0 ) not null,
location varchar(20) default 'Mumbai' not null
);
```

To insert record with default value omit the column in the insert statement as :

--> insert into Employee(empId, empName, contact, email, salary)

values(4, 'Chirag', 9956473883, 'chirag56@gmail.com', '32000.00');

If we insert a row without specifying location, it will default to 'Mumbai'.

If we add Not Null constraint with default constraint to any column and try to insert null values to that column it will occurs error as :

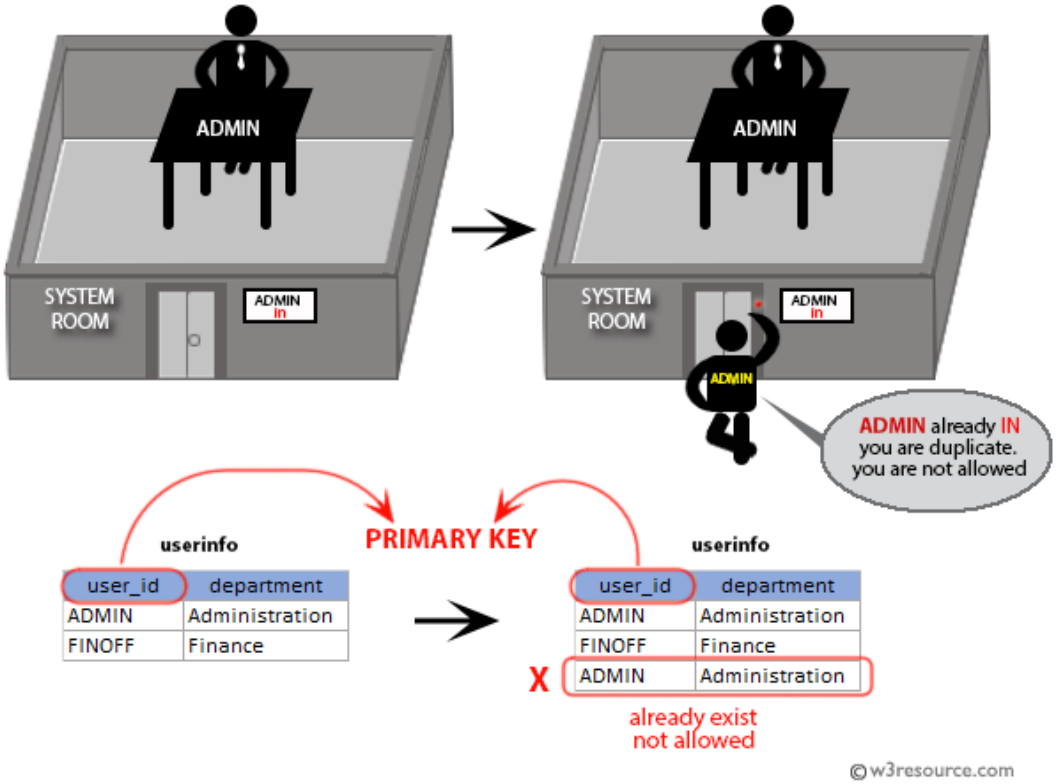
```
ERROR 1048 (23000): Column 'location' cannot be null
```

5. Primary key :

The PRIMARY KEY constraint uniquely identifies each record in a table.

Primary keys must contain UNIQUE values, and cannot contain NULL values.

A table can have only ONE primary key; and in the table, this primary key can consist of single or multiple columns.



=> Write a query to ensure that the empId column does not allow duplicate and null values.

```
create table Employee(empId int primary key,
empName varchar(50) not null,
contact bigint not null,
email varchar(100) Unique,
salary decimal(7,2) check (salary >= 0 ) not null,
location varchar(20) default 'Mumbai' not null
);
```

Insert one record in Employee table as :

--> insert into Employee values(1, 'Saya', 9956473883, 'saya768@gmail.com', '35000.00' , 'Bhiwandi');

Primary key value must be unique, so we can not insert the record with the same empId. Here, we will verify the constraint created on the empId column, by inserting records with duplicate empId values.

Insert second record with same empId as

--> insert into Employee values(1, 'Nikita', 9856473884, 'niki752@gmail.com', '30000.00' , 'Kalyan');

As we have mentioned above, if any column is defined as Primary Key in a database table, two records can not have the same value in that column. Therefore, the second insert statement generates the following error :

```
ERROR 1062 (23000): Duplicate entry '1' for key 'employee.PRIMARY'
```

Similarly, a primary key column cannot contain null values. Here, using the Insert statement we are passing a NULL value to the primary key column empId.

--> insert into Employee values(NULL, 'Nikita', 9856473884, 'niki752@gmail.com', '30000.00' , 'Kalyan');

This statement generates the following error :

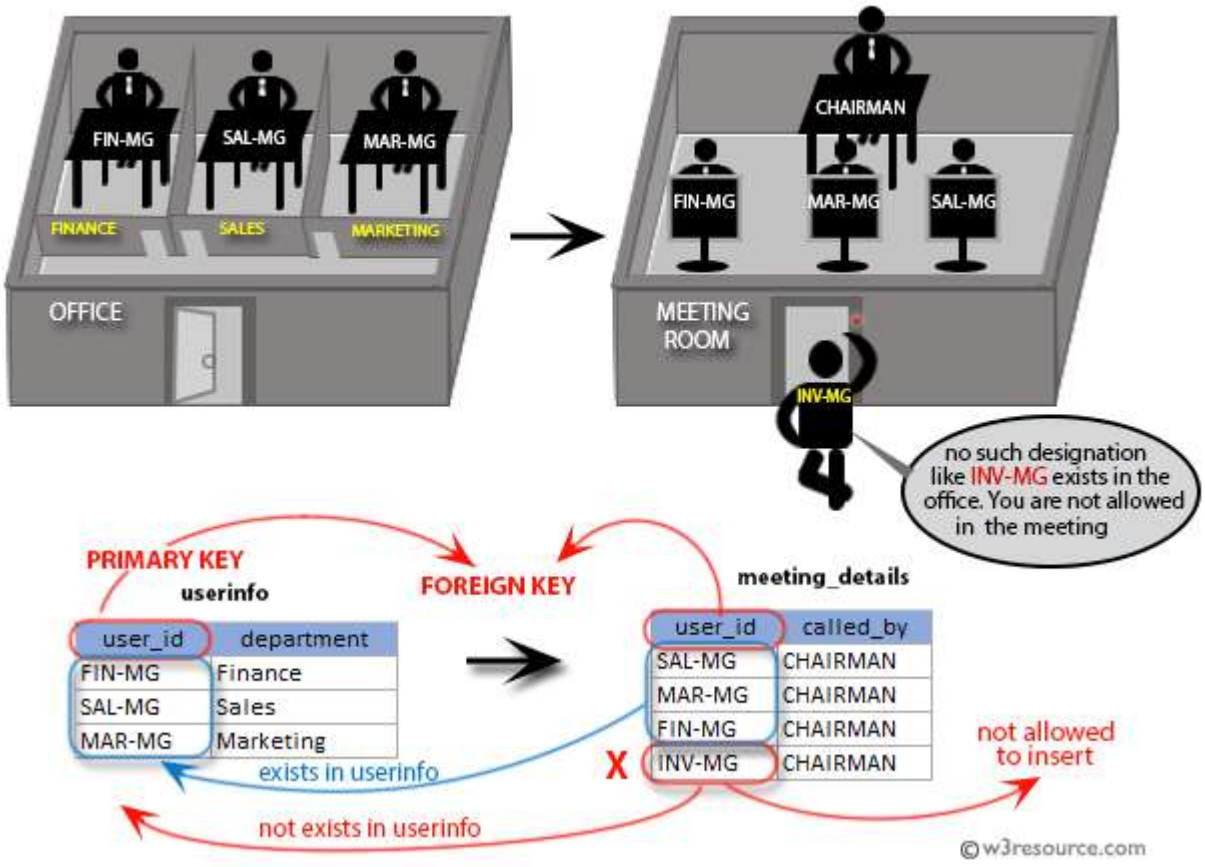
```
ERROR 1048 (23000): Column 'empId' cannot be null
```

6. Foreign Key :

The SQL FOREIGN KEY CONSTRAINT is used to ensure the referential integrity of the data in one table to match values in another table.

The FOREIGN KEY CONSTRAINT is a column or list of columns which points to the PRIMARY KEY of another table.

The main purpose of FOREIGN KEY is, only those values will appear which are present in the primary key table.



Create two tables Employee and Department. While creating tables , we add the foreign key constraint on column dept_id in Employee table reference on column dept_id of the Department table.

Department Table :

```
Create table Department(dept_id int primary key, dept_Name varchar(50));
```

Employee Table :

```
create table Employee(empId int primary key,
empName varchar(50) not null,
contact bigint not null,
email varchar(100) Unique,
salary decimal(7,2) check (salary >= 0 ) not null,
location varchar(20) default 'Mumbai' not null,
dept_id int, foreign key(dept_id) references Department(dept_id)
);
```

We have created a Foreign Key Constraint on a column named dept_id in the Employee table that references the column named dept_id of the Department table; so we can't drop table1 Department before dropping the Employee table.

To verify the foreign key if we try to drop Department table without deleting Employee table it will give error as :

```
ERROR 3730 (HY000): Cannot drop table 'Department' referenced by a
                    foreign key constraint 'Fk_dept_id' on table 'Employee'.
```

Data Definition Language

DDL is a part of SQL used for defining and managing all database objects such as tables, indexes, and schemas.

The main commands include:

- **CREATE**: Used to create a new database object, such as a table.
- **ALTER**: Used to modify an existing database object, like adding or changing a column.
- **DROP**: Used to delete a database object, such as a table.
- **TRUNCATE**: Used to remove all records from a table without deleting the table itself.

1. CREATE:

Used to create new database objects like tables, views, schemas, and databases.

Syntax :

```
CREATE TABLE table_name (
  column1 datatype,
  column2 datatype,
  column3 datatype,
 ....
 );
```

Simple Table Creation :

--> Create table Customers.

```
CREATE TABLE Customer (
    CustID INT,
    Name VARCHAR(100),
    Email VARCHAR(100),
    Phone VARCHAR(15),
    Address TEXT,
    CreatedAt DateTime
  );
```

Table Creating with Constraints.

```sql
CREATE TABLE Customer (
    UserID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Phone VARCHAR(15) NOT NULL,
    Address TEXT,
    COUNTRY VARCHAR(20) DEFAULT 'India'
    CreatedAt DateTime NOT NULL
);
```

2. ALTER:

The ALTER command in SQL is used to make changes to a table, view, or the entire database. We can add, modify, and drop constraints, columns, and indexes using the ALTER command in SQL.

Perform Different Types of Modifications using alter command for Employees table.

```
+-------+-------------+------+-----+---------+-------+
| Field | Type        | Null | Key | Default | Extra |
+-------+-------------+------+-----+---------+-------+
| Eid   | int         | YES  |     | NULL    |       |
| Ename | varchar(50) | YES  |     | NULL    |       |
+-------+-------------+------+-----+---------+-------+
```

---To add column in existing table.--

Example : Add Email column in Employees table.

```
mysql> Alter table Employees Add Email varchar(50);
```

Desc query to show structure of table. Here, Email column is added in the Employees table.

```
mysql> Desc Employees;
+-------+-------------+------+-----+---------+-------+
| Field | Type        | Null | Key | Default | Extra |
+-------+-------------+------+-----+---------+-------+
| Eid   | int         | YES  |     | NULL    |       |
| Ename | varchar(50) | YES  |     | NULL    |       |
| Email | varchar(50) | YES  |     | NULL    |       |
+-------+-------------+------+-----+---------+-------+
```

--To add the column after specific column--

Example : Add Address column after Ename column in Employees table.

```
mysql> Alter table Employees Add Address varchar(50) after Ename;
```

Here, Address column is added after Ename column in the Employees table.

```
mysql> Desc Employees;
+---------+-------------+------+-----+---------+-------+
| Field   | Type        | Null | Key | Default | Extra |
+---------+-------------+------+-----+---------+-------+
| Eid     | int         | YES  |     | NULL    |       |
| Ename   | varchar(50) | YES  |     | NULL    |       |
| Address | varchar(50) | YES  |     | NULL    |       |
| Email   | varchar(50) | YES  |     | NULL    |       |
+---------+-------------+------+-----+---------+-------+
```

--To add the column before specific column.--

Example : Add Contact column at the beginning in Employees table.

```
mysql> Alter table Employees Add Contact bigint first;
```

Here, Contact column is added in the beginning of Employees table.

```
mysql> Desc Employees;
+---------+-------------+------+-----+---------+-------+
| Field   | Type        | Null | Key | Default | Extra |
+---------+-------------+------+-----+---------+-------+
| Contact | bigint      | YES  |     | NULL    |       |
| Eid     | int         | YES  |     | NULL    |       |
| Ename   | varchar(50) | YES  |     | NULL    |       |
| Address | varchar(50) | YES  |     | NULL    |       |
| Email   | varchar(50) | YES  |     | NULL    |       |
+---------+-------------+------+-----+---------+-------+
```

--To add more than one column or multiple column in the table .--

Example : Add Salary and Age columns in the Employees table.

```
mysql> Alter table Employees Add Salary decimal(7,2), Add Location varchar(50);
```

Here, Salary and Age columns are added in the Employees table.

```
mysql> Desc Employees;
+----------+--------------+------+-----+---------+-------+
| Field    | Type         | Null | Key | Default | Extra |
+----------+--------------+------+-----+---------+-------+
| Contact  | bigint       | YES  |     | NULL    |       |
| Eid      | int          | YES  |     | NULL    |       |
| Ename    | varchar(50)  | YES  |     | NULL    |       |
| Address  | varchar(50)  | YES  |     | NULL    |       |
| Email    | varchar(50)  | YES  |     | NULL    |       |
| Salary   | decimal(7,2) | YES  |     | NULL    |       |
| Location | varchar(50)  | YES  |     | NULL    |       |
+----------+--------------+------+-----+---------+-------+
```

--To add more than one column or multiple columns after specific column in table .--

Example : Add Dept_id after Email and Mangaer_id column after Salary column in Employees table.

```
mysql> Alter table Employees Add Dept_id int after Email, Add Manager_id int after Salary;
```

Here, Dept_id column after Email and Manager_id after Salary is added in the Employees table.

```
mysql> Desc Employees;
+------------+--------------+------+-----+---------+-------+
| Field      | Type         | Null | Key | Default | Extra |
+------------+--------------+------+-----+---------+-------+
| Contact    | bigint       | YES  |     | NULL    |       |
| Eid        | int          | YES  |     | NULL    |       |
| Ename      | varchar(50)  | YES  |     | NULL    |       |
| Address    | varchar(50)  | YES  |     | NULL    |       |
| Email      | varchar(50)  | YES  |     | NULL    |       |
| Dept_id    | int          | YES  |     | NULL    |       |
| Salary     | decimal(7,2) | YES  |     | NULL    |       |
| Manager_id | int          | YES  |     | NULL    |       |
| Location   | varchar(50)  | YES  |     | NULL    |       |
+------------+--------------+------+-----+---------+-------+
```

--To change or rename column name in the table .--

Example : Change Contact column as a PhoneNo in Employees table.

```
mysql> Alter table Employees change column Contact PhoneNo bigint;
```

Here, Name of Contact column is changed as PhoneNo in the Employees table.

```
mysql> Desc Employees;
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| PhoneNo    | bigint      | YES  |     | NULL    |       |
| Eid        | int         | YES  |     | NULL    |       |
| Ename      | varchar(50) | YES  |     | NULL    |       |
| Address    | varchar(50) | YES  |     | NULL    |       |
| Email      | varchar(50) | YES  |     | NULL    |       |
| Dept_id    | int         | YES  |     | NULL    |       |
| Salary     | decimal(7,2)| YES  |     | NULL    |       |
| Manager_id | int         | YES  |     | NULL    |       |
| Location   | varchar(50) | YES  |     | NULL    |       |
+------------+-------------+------+-----+---------+-------+
```

--To re-arrange columns in table .--

Example : Arrange PhoneNo column after Email column in Employees table.

```
mysql> Alter table Employees Modify PhoneNo bigint after Email;
```

Here, PhoneNo column is arrange after Email column in the Employees table.

```
mysql> Desc Employees;
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| Eid        | int         | YES  |     | NULL    |       |
| Ename      | varchar(50) | YES  |     | NULL    |       |
| Address    | varchar(50) | YES  |     | NULL    |       |
| Email      | varchar(50) | YES  |     | NULL    |       |
| PhoneNo    | bigint      | YES  |     | NULL    |       |
| Dept_id    | int         | YES  |     | NULL    |       |
| Salary     | decimal(7,2)| YES  |     | NULL    |       |
| Manager_id | int         | YES  |     | NULL    |       |
| Location   | varchar(50) | YES  |     | NULL    |       |
+------------+-------------+------+-----+---------+-------+
```

--To rename table name .--

Example : Raname the table name of Employees as EmpDetails.

```
mysql> Alter table Employees rename EmpDetails;
```

Here, Table name changed from Employees to EmpDetails.

--To change datatype of column in table.--

Example : Change the datatype of column PhoneNo in Employees table.

```
mysql> Alter table Employees Modify PhoneNo varchar(20);
```

Here, Datatype of PhoneNo is changed from bigint to varchar(20) in the Employees table.

```
mysql> Desc Employees;
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| Eid        | int         | YES  |     | NULL    |       |
| Ename      | varchar(50) | YES  |     | NULL    |       |
| Address    | varchar(50) | YES  |     | NULL    |       |
| Email      | varchar(50) | YES  |     | NULL    |       |
| PhoneNo    | varchar(20) | YES  |     | NULL    |       |
| Dept_id    | int         | YES  |     | NULL    |       |
| Salary     | decimal(7,2)| YES  |     | NULL    |       |
| Manager_id | int         | YES  |     | NULL    |       |
| Location   | varchar(50) | YES  |     | NULL    |       |
+------------+-------------+------+-----+---------+-------+
```

--To apply constraint to column of existing table.--

Example : Ensure that the Ename column cannot be null in Employees table.

```
mysql> Alter table Employees Modify Ename varchar(100) Not Null;
```

Here, PhoneNo column is arrange after Email column in the Employees table.

```
mysql> Desc Employees;
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| Eid        | int         | YES  |     | NULL    |       |
| Ename      | varchar(100)| NO   |     | NULL    |       |
| Address    | varchar(50) | YES  |     | NULL    |       |
| Email      | varchar(50) | YES  |     | NULL    |       |
| PhoneNo    | varchar(20) | YES  |     | NULL    |       |
| Dept_id    | int         | YES  |     | NULL    |       |
| Salary     | decimal(7,2)| YES  |     | NULL    |       |
| Manager_id | int         | YES  |     | NULL    |       |
| Location   | varchar(50) | YES  |     | NULL    |       |
+------------+-------------+------+-----+---------+-------+
```

--To drop column in the table .--

Example : Drop the column Manager_id from Employees table.

```
mysql> Alter table Employees Drop Manager_id;
```

Here, Manager_id column is deleted from Employees table.

```
mysql> Desc Employees;
+----------+--------------+------+-----+---------+-------+
| Field    | Type         | Null | Key | Default | Extra |
+----------+--------------+------+-----+---------+-------+
| Eid      | int          | YES  |     | NULL    |       |
| Ename    | varchar(100) | NO   |     | NULL    |       |
| Address  | varchar(50)  | YES  |     | NULL    |       |
| Email    | varchar(50)  | YES  |     | NULL    |       |
| PhoneNo  | varchar(20)  | YES  |     | NULL    |       |
| Dept_id  | int          | YES  |     | NULL    |       |
| Salary   | decimal(7,2) | YES  |     | NULL    |       |
| Location | varchar(50)  | YES  |     | NULL    |       |
+----------+--------------+------+-----+---------+-------+
```

--To add Primary key to column of existing table by alter command.--

Example : Add Primary key to column Eid in Employees table.

```
mysql> Alter table Employees Add primary key(Eid);
```

Here, Primary key is added to Eid column.

```
mysql> Desc Employees;
+----------+--------------+------+-----+---------+-------+
| Field    | Type         | Null | Key | Default | Extra |
+----------+--------------+------+-----+---------+-------+
| Eid      | int          | NO   | PRI | NULL    |       |
| Ename    | varchar(100) | NO   |     | NULL    |       |
| Address  | varchar(50)  | YES  |     | NULL    |       |
| Email    | varchar(50)  | YES  |     | NULL    |       |
| PhoneNo  | varchar(20)  | YES  |     | NULL    |       |
| Dept_id  | int          | YES  |     | NULL    |       |
| Salary   | decimal(7,2) | YES  |     | NULL    |       |
| Location | varchar(50)  | YES  |     | NULL    |       |
+----------+--------------+------+-----+---------+-------+
```

--To Add Foreign Key to column of existing table by alter.--

To add foreign key we required two tables in which Foreign Key is a column in one table that matches a Primary Key in another table.

Assume, the Employees and Department tables have already been created in the SQL database. Now, we will add a Foreign Key Constraint on the dept_id column of the Employees table.

Example : Add Foreign key constraint to the Employees table.

```
mysql> Alter table Employees Add constraint fk_deptId foreign key(dept_id) references Department(dept_Id);
```

Here, Foreign key is add in the Emloyees table.

```
mysql> Desc Employees;
+----------+--------------+------+-----+---------+-------+
| Field    | Type         | Null | Key | Default | Extra |
+----------+--------------+------+-----+---------+-------+
| Eid      | int          | NO   | PRI | NULL    |       |
| Ename    | varchar(100) | NO   |     | NULL    |       |
| Address  | varchar(50)  | YES  |     | NULL    |       |
| Email    | varchar(50)  | YES  |     | NULL    |       |
| PhoneNo  | varchar(20)  | YES  |     | NULL    |       |
| Dept_id  | int          | YES  | MUL | NULL    |       |
| Salary   | decimal(7,2) | YES  |     | NULL    |       |
| Location | varchar(50)  | YES  |     | NULL    |       |
+----------+--------------+------+-----+---------+-------+
```

```
mysql> Desc Department;
+-----------+-------------+------+-----+---------+-------+
| Field     | Type        | Null | Key | Default | Extra |
+-----------+-------------+------+-----+---------+-------+
| dept_Id   | int         | NO   | PRI | NULL    |       |
| dept_name | varchar(20) | YES  |     | NULL    |       |
+-----------+-------------+------+-----+---------+-------+
```

Truncate Command :

The TRUNCATE command is used to remove all of the rows from a table, regardless of whether or not any conditions are met.

Example :

-->Delete All records from EmpDetails table.

Employee table with records.

```
mysql> select * from EmpDetails;
+-----+-------+-------------------+----------+
| Eid | Ename | Email             | Salary   |
+-----+-------+-------------------+----------+
|   1 | Saya  | saya13@gmail.com  | 35000.00 |
|   2 | Chaya | chaya13@gmail.com | 30000.00 |
|   3 | Jaya  | jaya13@gmail.com  | 28000.00 |
+-----+-------+-------------------+----------+
```

Following SQL TRUNCATE table EmpDetails statement will remove all the records of the EmpDetails table –

```
mysql> Truncate table EmpDetails;
```

Now, the Employee table is truncated and the output from SELECT statement will be as shown in below –

```
mysql> select * from EmpDetails;
Empty set (0.00 sec)
```

It deleted all records from the EmpDetails table but structure of EmpDetails remain as it is as shown in the below :

```
mysql> desc EmpDetails;
+--------+--------------+------+-----+---------+-------+
| Field  | Type         | Null | Key | Default | Extra |
+--------+--------------+------+-----+---------+-------+
| Eid    | int          | NO   | PRI | NULL    |       |
| Ename  | varchar(50)  | YES  |     | NULL    |       |
| Email  | varchar(30)  | YES  |     | NULL    |       |
| Salary | decimal(7,2) | YES  |     | NULL    |       |
+--------+--------------+------+-----+---------+-------+
```

Drop Command :

The DELETE command in SQL removes one or more rows from a table based on the conditions specified in a WHERE Clause.

Example :

--> Delete the EmpDetails Table from the database.

```
mysql> Drop table EmpDetails;
```

Now, to verify if the table is actually dropped, we can use the Desc EmpDetails command as shown –

```
mysql> Desc EmpDetails;
```

Following error is displayed –

```
mysql> Desc EmpDetails;
ERROR 1146 (42S02): Table 'medicinedeliverydatabase.empdetails' doesn't exist
```

Data Manipulation Language

DML is a part of SQL used to manipulate and manage the data stored in a database.

The main commands include:

- INSERT: Used to add new records into a table.
- UPDATE: Used to modify existing records in a table.
- DELETE: Used to remove specific records from a table.

INSERT :

The SQL INSERT INTO Statement is used to add new rows of data into a table in the database. It allows you to insert one or more rows of data into a specified table.

Basic Syntax :

There are two basic syntaxes of the SQL INSERT into statement which are shown below :

```
INSERT INTO TABLE_NAME (column1, column2...columnN)
VALUES (value1, value2...valueN);
```

Here, column1, column2, column3,...columnN are the names of the columns in the table into which you want to insert the data.

There is another syntax of INSERT Into statement where you can specify only column values without column names. But, make sure the order of the values is in the same order as the columns in the table.

Following is second syntax of the INSERT Query –

```
INSERT INTO TABLE_NAME
VALUES (value1,value2...valueN);
```

Example : Add Given data to the Medicines Table.

Id	Name	Brand	Price	Expiry Date
1	Paracetamol	Tylenol	5.99	2026-08-15
2	Aspirin	Bayer	4.99	2027-04-20
3	Cetirizine	Zyrtec	8.00	2026-11-05
4	Metformin	Glucophage	10.99	2026-09-30
5	Omeprazole	Prilosec	9.49	2025-07-12

To insert given data in Medicines table, first we have to create a table with name Medicines in the MySQL database using the CREATE TABLE statement as shown below

```
CREATE TABLE Medicines (
    Id INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    Brand VARCHAR(100),
    Price DECIMAL(10,2) NOT NULL,
    Expiry_date DATE NOT NULL
  );
```

The following INSERT into statements will create 2 records in the empty Medicines table.

```
INSERT into Medicines (Id, Name, Brand, Price, Expiry_date)
VALUES (1, 'Paracetamol', 'Tylenol', 2.99, '2026-08-15' );

INSERT into Medicines (Id, Name, Brand, Price, Expiry_date)
VALUES (2, 'Aspirin', 'Bayer', 4.99, '2027-04-20');
```

We can also insert multiple rows at once using the following query as shown below –

```
INSERT into Medicines (Id, Name, Brand, Price, Expiry_date) VALUES
(3, ' Cetirizine ', ' Zyrtec ', 8.00, '2026-11-05'),
(4, ' Metformin ', ' Glucophage ', 10.99, '2026-09-30');
```

Following query adds another record in the Medicines table using the second syntax as shown below –

```
INSERT into Medicines VALUES (5, ' Omeprazole ', ' Prilosec ', 9.49, '2025-07-12');
```

To check if the records are inserted into the Medicines table, we use the SELECT query

```
mysql> select * from Medicines;
+----+-------------+------------+-------+-------------+
| Id | Name        | Brand      | Price | Expiry_date |
+----+-------------+------------+-------+-------------+
|  1 | Paracetamol | Tylenol    |  2.99 | 2026-08-15  |
|  2 | Aspirin     | Bayer      |  4.99 | 2027-04-20  |
|  3 | Cetirizine  | Zyrtec     |  8.00 | 2026-11-05  |
|  4 | Metformin   | Glucophage | 10.99 | 2026-09-30  |
|  5 | Omeprazole  | Prilosec   |  9.49 | 2025-07-12  |
+----+-------------+------------+-------+-------------+
```

Update :

The UPDATE statement in SQL is used to modify existing records (rows) in a table. It allows you to update one or more columns in a table with new values based on specified conditions.

It only modifies the data present in a table without affecting the table's structure.

To filter records that needs to be modified, you can use a WHERE clause with UPDATE statement. Using a WHERE clause, you can either update a single row or multiple rows.

- UPDATE : It specifies the table name where the update operation will be performed.
- SET : It specifies the columns to be updated along with their new values.
- WHERE : It sspecifies the condition that determines which rows will be updated. If omitted, all rows in the table will be updated.

Basic Syntax :

The basic syntax of the SQL UPDATE statement with a WHERE clause is as follows –

```
UPDATE table_name
SET column1 = value1, column2 = value2,..., columnN = valueN
WHERE [condition];
```

Example :

In the given Medicines table, Change the Brand name as 'Alerid' for the medicine 'Cetirizine' whose id number is 3.

Id	Name	Brand	Price	Expiry Date
1	Paracetamol	Tylenol	2.99	2026-08-15
2	Aspirin	Bayer	4.99	2027-04-20
3	Cetirizine	Zyrtec	8.00	2026-11-05
4	Metformin	Glucophage	10.99	2026-09-30
5	Omeprazole	Prilosec	9.49	2025-07-12

The following query will update the Brand name from 'Zyrtec' to 'Alerid' for the medicine 'Cetirizine'.

```
UPDATE Medicines SET Brand = 'Alerid' WHERE Id = 3;
```

To verify whether the records of the table are modified or not, we use the following SELECT query –

```
SELECT * FROM Medicines WHERE Id=3;
```

Now, the Medicines table would have the following record –

```
mysql> SELECT * FROM Medicines WHERE Id=3;
+----+------------+--------+-------+-------------+
| Id | Name       | Brand  | Price | Expiry_date |
+----+------------+--------+-------+-------------+
|  3 | Cetirizine | Alerid |  8.00 | 2026-11-05  |
+----+------------+--------+-------+-------------+
```

Update Multiple Columns :

Change the Price = '7.99' and ExpiryDate='2027-03-17' for medicine whose id is 4.

```
UPDATE Medicines SET Price = '7.99', Expiry_date='2027-03-17'  WHERE Id = 4;
```

Here, Price and Expirydate columns values are updated for Metformin medicine.

```
mysql> SELECT * FROM Medicines WHERE Id=4;
+----+-----------+------------+-------+-------------+
| Id | Name      | Brand      | Price | Expiry_date |
+----+-----------+------------+-------+-------------+
|  4 | Metformin | Glucophage |  7.99 | 2027-03-17  |
+----+-----------+------------+-------+-------------+
```

Update Multiple Rows :

Change the Price of medicine as Price=6.49 whose price are less than 5.

```
UPDATE Medicines SET Price = '6.49' WHERE Price<=5;
```

Here, Price of 2 rows are updated by given condition in medicines table.

```
mysql> SELECT * FROM Medicines;
+----+-------------+------------+-------+-------------+
| Id | Name        | Brand      | Price | Expiry_date |
+----+-------------+------------+-------+-------------+
|  1 | Paracetamol | Tylenol    |  6.49 | 2026-08-15  |
|  2 | Aspirin     | Bayer      |  6.49 | 2027-04-20  |
|  3 | Cetirizine  | Alerid     |  8.00 | 2026-11-05  |
|  4 | Metformin   | Glucophage |  7.99 | 2027-03-17  |
|  5 | Omeprazole  | Prilosec   |  9.49 | 2025-07-12  |
+----+-------------+------------+-------+-------------+
```

DELETE :

The DELETE Statement is used to delete the records from an existing table. In order to delete particular records, we need to use the WHERE clause along with the DELETE statement.

If you execute DELETE statement without a WHERE clause, it will delete all the records from the table.

Basic Syntax :

The basic syntax of the SQL DELETE Query with the WHERE clause is as follows –

```
DELETE FROM table_name WHERE [condition];
```

Example :

From given Medicines table, delete the record of medicine whose id is 4.

Id	Name	Brand	Price	Expiry Date
1	Paracetamol	Tylenol	2.99	2026-08-15
2	Aspirin	Bayer	4.99	2027-04-20
3	Cetirizine	Zyrtec	8.00	2026-11-05
4	Metformin	Glucophage	10.99	2026-09-30
5	Omeprazole	Prilosec	9.49	2025-07-12

The following query deletes the record of a Medicines, whose ID is 4.

```
DELETE FROM Medicines WHERE Id = 4;
```

To verify whether the record have been deleted from the table, we need to retrieve the modified table using the SELECT query.

Now, the Medicines table would have the following records −

```
mysql> SELECT * FROM Medicines;
+----+-------------+----------+-------+-------------+
| Id | Name        | Brand    | Price | Expiry_date |
+----+-------------+----------+-------+-------------+
|  1 | Paracetamol | Tylenol  |  6.49 | 2026-08-15  |
|  2 | Aspirin     | Bayer    |  6.49 | 2027-04-20  |
|  3 | Cetirizine  | Alerid   |  8.00 | 2026-11-05  |
|  5 | Omeprazole  | Prilosec |  9.49 | 2025-07-12  |
+----+-------------+----------+-------+-------------+
```

Data Query Language

DQL is a subset of SQL used to query and retrieve data from a database.

The primary command in DQL is:

- SELECT: Used to retrieve data from one or more tables based on specified criteria.

SELECT :

The SELECT statement is a fundamental query in SQL (Structured Query Language) used to retrieve data from a database.

It allows you to specify which columns to retrieve and from which table or tables.

Important Points with SQL SELECT Statement

- It is used to access records from one or more database tables and views.
- The SELECT statement retrieves selected data based on specified conditions.
- The result of a SELECT statement is stored in a result set or result table.
- The SELECT statement can be used to access specific columns or all columns from a table.
- It can be combined with clauses like WHERE, GROUP BY, HAVING, and ORDER BY for more refined data retrieval.
- The SELECT statement is versatile and allows users to fetch data based on various criteria efficiently.

Example : Perform different types of queries for given Customers table using select statement with clauses.

Id	FirstName	LastName	Email	Phone No	City	Age
1	Saya	Suryvanshi	saya768@gmail.com	9856237414	Bhiwandi	25
2	Nikita	Giri	niki222@gmail.com	8856237415	Kalyan	23
3	Parth	Shirke	parth257@gmail.com	9986237454	Kalyan	22
4	Chirag	Kamble	chirag777@gmail.com	8256237369	Ambarnath	25
5	Ubaid	Fazlani	ubaid933@gmail.com	9556237347	Bhiwandi	24
6	Dhiraj	Anand	dhiraj768@gmail.com	8956237411	Dombivali	23

***Retrieve Data using SELECT Query :

Syntax :

```
SELECT column1,column2…. FROM table_name ;
```

--> Fetch FirstName, LastName from the table Customers:

```
SELECT FirstName, LastName FROM Customers;
```

Output :

```
mysql> SELECT FirstName, LastName FROM Customers;
+-----------+-------------+
| FirstName | LastName    |
+-----------+-------------+
| Saya      | Suryavanshi |
| Nikita    | Giri        |
| Parth     | Shirke      |
| Chirag    | Kamble      |
| Ubaid     | Fazlani     |
| Dhiraj    | Anand       |
+-----------+-------------+
```

***Fetch All table using SELECT Statement :

Syntax :

```
SELECT * FROM table_name;
```

--> Retrieve all records of Customers table.

```
SELECT * FROM Customers;
```

Output :

```
mysql> SELECT * FROM Customers;
+----+-----------+-------------+---------------------+------------+-----------+-----+
| Id | FirstName | LastName    | Email               | Phone      | City      | Age |
+----+-----------+-------------+---------------------+------------+-----------+-----+
|  1 | Saya      | Suryavanshi | saya768@gmail.com   | 9856237414 | Bhiwandi  |  25 |
|  2 | Nikita    | Giri        | niki222@gmail.com   | 8856237415 | Kalyan    |  23 |
|  3 | Parth     | Shirke      | parth257@gmail.com  | 9986237454 | Kalyan    |  22 |
|  4 | Chirag    | Kamble      | chirag777@gmail.com | 8256237369 | Ambarnath |  25 |
|  5 | Ubaid     | Fazlani     | ubaid933@gmail.com  | 9556237347 | Bhiwandi  |  24 |
|  6 | Dhiraj    | Anand       | dhiraj768@gmail.com | 8956237411 | Dombivali |  23 |
+----+-----------+-------------+---------------------+------------+-----------+-----+
```

***SELECT Statement with WHERE Clause

- The **WHERE** clause in SQL is used to filter records based on specified conditions.
- It allows you to retrieve rows that meet certain criteria from a table.

Syntax of WHERE Clause:

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

--> Retrieve FirstName, LastName and Age of Customers whose age is 25.

```
SELECT FirstName, LAstName, Age FROM Customers where Age = '25';
```

Output :

```
+-----------+-------------+-----+
| FirstName | LastName    | Age |
+-----------+-------------+-----+
| Saya      | Suryavanshi |  25 |
| Chirag    | Kamble      |  25 |
+-----------+-------------+-----+
```

***SELECT Statement with GROUP BY Clause

The SQL **GROUP BY** clause is used in conjunction with the SELECT statement to arrange identical data into groups. This clause follows the WHERE clause in a SELECT statement and precedes the ORDER BY and HAVING clauses (if they exist).

The main purpose of grouping the records of a table based on particular columns is to perform calculations on these groups. Therefore, The GROUP BY clause is typically used with aggregate functions such as SUM(), COUNT(), AVG(), MAX(), or MIN() etc.

Syntax :

```
SELECT column_name1, column_name2.....,
FROM table_name
GROUP BY column_name;
```

--> Group the customers based on AGE and counts the number of records in each group.

```
SELECT Age, COUNT(FirstName) FROM CUSTOMERS GROUP BY Age;
```

Output :

```
+-----+------------------+
| Age | COUNT(FirstName) |
+-----+------------------+
|  25 |                2 |
|  23 |                2 |
|  22 |                1 |
|  24 |                1 |
+-----+------------------+
```

***SELECT Statement with HAVING Clause

The SQL HAVING clause is similar to the WHERE clause; both are used to filter rows in a table based on specified criteria. However, the HAVING clause is used to filter grouped rows instead of single rows. These rows are grouped together by the GROUP BY clause, so, the HAVING clause must always be followed by the GROUP BY clause.

Moreover, the HAVING clause can be used with aggregate functions such as COUNT(), SUM(), AVG(), etc., whereas the WHERE clause cannot be used with them.

Syntax :

```
SELECT column1, column2, aggregate_function(column)
FROM table_name
GROUP BY column1, column2
HAVING condition;
```

The following code block shows the position of the HAVING Clause in a query

```
SELECT
FROM
WHERE
GROUP BY
HAVING
ORDER BY
```

--> Find cities where customers have placed more than 2 orders in total.

```sql
SELECT City, SUM(Orders) AS 'TotalOrderCount'
FROM Customers
GROUP BY City
HAVING SUM(Orders) > 2;
```

Output :

City	TotalOrderCount
Bhiwandi	4
Kalyan	5
Ambarnath	5
Dombivali	5

*****SELECT Statement with ORDER BY clause in SQL**

The SQL **ORDER BY** clause is used to sort the data in either ascending or descending order, based on one or more columns.

In MySql, order by clause sort data by default in ascending order.

Syntax :

```sql
SELECT column-list
FROM table_name
[ORDER BY column1, column2, .. columnN] [ASC | DESC];
```

--> Fetch the FirstName and Age of Customers in descending order based on the FirstName.

```sql
SELECT FirstName,  Age FROM Customers ORDER BY FirstName DESC;
```

Output :

FirstName	Age
Ubaid	24
Saya	25
Parth	22
Nikita	23
Dhiraj	23
Chirag	25

Operators

Arithmatic Operators :

The SQL Arithmetic operators are used to perform basic mathematical operations on numeric data types in a database.

The most common arithmetic operators used in SQL are:

Operator	Description	Example
+	Adds the values both sides of the operator.	10 + 20 gives 30
-	Subtracts right hand operand from left hand operand.	20 - 30 gives 10
*	Multiplies the values from both sides of the operator.	10 * 20 gives 200
/	Divides left hand operand by right hand operand.	20 / 10 gives 2
%	Divides left hand operand by right hand operand and returns the reminder remainder.	5 % 2 gives 1

Perform Different types of operations using operators on given Customers table.

Id	Name	LastName	City	Age	Orders
1	Saya	Suryvanshi	Bhiwandi	25	2
2	Nikita	Giri	Kalyan	23	5
3	Parth	Shirke	Kalyan	22	3
4	Chirag	Kamble	Ambarnath	25	6
5	Ubaid	Fazlani	Bhiwandi	24	5
6	Dhiraj	Anand	Dombivali	23	4

1. Addition (+) Operator :

The addition operator is used to add two or more numeric values. It is similar to the "plus" symbol in basic mathematics.

--> Increase Total Orders by 2 for Each Customer.

```sql
SELECT FirstName, Orders, Orders + 2 AS UpdatedOrders FROM Customers;
```

Output :

FirstName	Orders	UpdatedOrders
Saya	2	4
Nikita	3	5
Parth	2	4
Chirag	5	7
Ubaid	2	4
Dhiraj	5	7

2. Subtraction (-) Operator :

The subtraction operator in SQL is used to subtract one numeric value from another. It is similar to the "minus" symbol in basic mathematics.

--> Decrease Total Orders by 1.

```sql
SELECT FirstName, Orders, Orders - 1 AS UpdatedOrders FROM Customers;
```

Output :

FirstName	Orders	UpdatedOrders
Saya	2	1
Nikita	3	2
Parth	2	1
Chirag	5	4
Ubaid	2	1
Dhiraj	5	4

3. Multiplication (*) Operator :

The multiplication operator in SQL is used to perform mathematical multiplication on numeric values. It allows us to multiply two or more columns or numeric expressions together, resulting in a new value representing the product of the operands.

--> Double the Total Orders of customers.

```sql
SELECT FirstName, Orders, Orders * 2 AS UpdatedOrders FROM Customers;
```

Output :

FirstName	Orders	UpdatedOrders
Saya	2	4
Nikita	3	6
Parth	2	4
Chirag	5	10
Ubaid	2	4
Dhiraj	5	10

4. Division (/) Operator :

The division operator (/) in SQL is used to perform mathematical division on numeric values. It allows us to divide one numeric operand by another, resulting in a new value representing the quotient of the division.

--> Divide Orders by 2 to Get Half Orders.

```sql
SELECT FirstName, Orders, Orders / 2 AS HalfOrders FROM Customers;
```

Output :

FirstName	Orders	HalfOrders
Saya	2	1.0000
Nikita	3	1.5000
Parth	2	1.0000
Chirag	5	2.5000
Ubaid	2	1.0000
Dhiraj	5	2.5000

5. Modulus (%) Operator :

The modulus operator (%) in SQL is used to find the remainder after dividing one numeric value by another. It returns the integer remainder of the division operation.

--> Find out the Customers Id and FirstName who have an even Customer ID.

```
SELECT Id, FirstName FROM Customers WHERE ID % 2 = 0;
```

Output :

```
+----+-----------+
| Id | FirstName |
+----+-----------+
|  2 | Nikita    |
|  4 | Chirag    |
|  6 | Dhiraj    |
+----+-----------+
```

Comparison Operator :

The SQL comparison operators are used to compare two or more expressions and return a Boolean value (True or False) based on the comparison result.

Some commonly used SQL comparison operators:

Operator	Description	Example
=	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(a = b) is not true.
!=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(a != b) is true.
<>	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(a <> b) is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(a > b) is not true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(a < b) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(a >= b) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(a <= b) is true.

1. Equal to (=) Operator :

The equal to operator in SQL is used to check if two values are equal. It is used to filter rows with values that matches a specific condition.

-->Retrieve FirstName and Age of customers with age equal to 23.

```
SELECT FirstName, Age FROM customers WHERE age = 23;
```

Output :

```
+-----------+-----+
| FirstName | Age |
+-----------+-----+
| Nikita    |  23 |
| Dhiraj    |  23 |
+-----------+-----+
```

2. Not Equal to (=) operator :

The not equal to operator in SQL checks if two expressions are not equal. It is used to filter rows with values that do not match a specific condition.

-->Retrieve FirstName and Age of customers with age is not equal to 25.

```
SELECT FirstName, Age FROM customers WHERE Age != 25;
```

Output :

```
+-----------+-----+
| FirstName | Age |
+-----------+-----+
| Nikita    |  23 |
| Parth     |  22 |
| Ubaid     |  24 |
| Dhiraj    |  23 |
+-----------+-----+
```

3. Greater than(>) Operator :

The greater than operator in SQL is used to return rows having values greater than the specified value. It checks if the left expression is greater than the right expression. It is used to filter rows with values above a certain threshold.

-->Retrieve FirstName and Age of customers with age is greater than 24.

```
SELECT FirstName, Age FROM customers WHERE age > 24;
```

Output :

```
+-----------+-----+
| FirstName | Age |
+-----------+-----+
| Saya      |  25 |
| Chirag    |  25 |
+-----------+-----+
```

4. Less than (<) Operator :

The less than operator in SQL is used to return rows having values less than the specified value. It checks if the left expression is less than the right expression. It is used to filter rows with values below a certain threshold.

-->Retrieve FirstName and Age of customers with age is less than 23.

```
SELECT FirstName, Age FROM customers WHERE age < 23;
```

Output :

```
+-----------+-----+
| FirstName | Age |
+-----------+-----+
| Parth     |  22 |
+-----------+-----+
```

5. Greater than or equal to (>=) Operator :

The greater than or equal to operator in SQL is used to return rows having values either greater than or equal to the specified value. It checks if the left expression is greater than or equal to the right expression. It is used to filter rows with values equal to or above a certain limit.

-->Retrieve FirstName and Age of customers with age is greater than or equal to 24.

```
SELECT FirstName, Age FROM customers WHERE age >= 24;
```

Output :

```
+-----------+-----+
| FirstName | Age |
+-----------+-----+
| Saya      |  25 |
| Chirag    |  25 |
| Ubaid     |  24 |
+-----------+-----+
```

6. Less than or equal to (<=) Operator :

The less than or equal to operator in SQL is used to return rows having values either less than or equal to the specified value. It checks if the left expression is less than or equal to the right expression. It is used to filter rows with values equal to or below a certain limit.

-->Retrieve FirstName and Age of customers with age is less than or equal to 24.

```
SELECT FirstName, Age FROM customers WHERE age <= 24;
```

Output :

```
+-----------+-----+
| FirstName | Age |
+-----------+-----+
| Nikita    |  23 |
| Parth     |  22 |
| Ubaid     |  24 |
| Dhiraj    |  23 |
+-----------+-----+
```

Logical Operator :

logical operators are used to create conditional expressions that evaluates to either true or false. They are used in the WHERE clause of SELECT, UPDATE, DELETE, and other SQL statements to filter data based on specified conditions.

Some of logical operators

Operator	Description
AND	TRUE if all the conditions separated by AND are TRUE.
OR	TRUE if any of the conditions separated by OR is TRUE
NOT	Reverses the value of any other Boolean operator.

1. AND Operator :

The SQL AND operator is used to combine multiple conditions in a WHERE clause or a HAVING clause. It allows you to retrieve rows that meet all specified conditions. If all the conditions joined by AND are true for a row, that row will be included in the result set.

--> Fetch the FirstName and Age of the customers whose Age is greater than 23 and less than 25 years.

```
SELECT FirstName, Age FROM customers WHERE Age > 23 AND Age < 25;
```

Output :

```
+-----------+-----+
| FirstName | Age |
+-----------+-----+
| Ubaid     |  24 |
+-----------+-----+
```

2. OR Operator :

The SQL OR operator is used to combine multiple conditions in a WHERE clause or a HAVING clause. It allows us to retrieve rows that meet at least one of the specified conditions. If any of the conditions joined by the OR operator are true for a row, that row will be included in the result set.

--> Fetch the FirstName and Age of the customers whose Age is less than 23 or greater than 24 years.

```
SELECT FirstName, Age FROM customers WHERE Age < 23 OR Age > 24;
```

Output :

```
+-----------+-----+
| FirstName | Age |
+-----------+-----+
| Saya      |  25 |
| Parth     |  22 |
| Chirag    |  25 |
+-----------+-----+
```

3. NOT Operator :

The SQL NOT operator used to negate a condition. It returns true if the specified condition is false and false if the condition is true. The NOT operator is commonly used to perform negative or inverse filtering in SQL queries.

--> Fetch the FirstName and Age of the customers whose Age is not greater than 23 years.

```
SELECT FirstName, Age FROM customers WHERE Not (Age < 23);
```

Output :

```
+-----------+-----+
| FirstName | Age |
+-----------+-----+
| Nikita    |  23 |
| Parth     |  22 |
| Dhiraj    |  23 |
+-----------+-----+
```

Range Operator :

The range operator in SQL is used to specify a range of values within a certain criteria.

1. BETWEEN Operator :

BETWEEN operator is used to retrieve the data within a specified range. The retrieved values can be integers, characters, or dates.

--> Retrieve the FirstName and Age of the Customers whose Age is between 20 and 23.

```
SELECT FirstName, Age FROM customers WHERE Age BETWEEN 20 AND 23;
```

Output :

```
+-----------+-----+
| FirstName | Age |
+-----------+-----+
| Nikita    |  23 |
| Parth     |  22 |
| Dhiraj    |  23 |
+-----------+-----+
```

2. NOT BETWEEN Operator :

The NOT BETWEEN operator in SQL works exactly opposite to BETWEEN operator. This is used to retrieve the data which is not present in the specified range.

--> Retrieve the FirstName and Age of the Customers whose Age is between 20 and 24.

```
SELECT FirstName, Age FROM customers WHERE Age NOT BETWEEN 22 AND 24;
```

Output :

```
+-----------+-----+
| FirstName | Age |
+-----------+-----+
| Saya      |  25 |
| Chirag    |  25 |
+-----------+-----+
```

List Operator

The list operator in SQL allows you to filter data based on a list of specified values.

1. IN Operator :

IN Operator is used to specify multiple values or sub query in the WHERE clause. It returns all rows in which the specified column matches one of the values in the list.

--> Retrieve the FirstName and Age of the Customers with age '22' or '26'

```
SELECT FirstName, Age FROM Customers WHERE Age IN (22, 26);
```

Output :

```
+-----------+-----+
| FirstName | Age |
+-----------+-----+
| Parth     |  22 |
+-----------+-----+
```

2. NOT IN Operator :

The SQL IN operator can be used in combination with the NOT operator to exclude specific values in a WHERE clause. Checks if a value does not match any value in a specified list.

--> Retrieve the FirstName and Age of the Customers where age is not equal to '22' or '26'

```
SELECT FirstName, Age FROM Customers WHERE Age NOT IN (23, 25);
```

Output :

```
+-----------+-----+
| FirstName | Age |
+-----------+-----+
| Parth     |  22 |
| Ubaid     |  24 |
+-----------+-----+
```

LIKE Operator :

The LIKE operator is used to retrieve the data in a column of a table, based on a specified pattern.

It is used along with the WHERE clause to filter the rows based on the given pattern. These patterns are specified using Wildcards.

Wildcards are special characters used in SQL queries to match patterns in the data. Following are the wildcards used in conjunction with the LIKE operator.

WildCard	Definition
%	The percent sign represents zero, one or multiple characters.
_	The underscore represents a single number or character.

1. % Wildcard :

--> Find the FirstName, City and Age of the Customers where FirstName starts 'S'.

```
SELECT FirstName, City, Age FROM Customers WHERE FirstName LIKE 'S%';
```

Output :

```
+-----------+----------+-----+
| FirstName | City     | Age |
+-----------+----------+-----+
| Saya      | Bhiwandi |  25 |
+-----------+----------+-----+
```

2. _ Wildcard :

--> Find the FirstName, City and Age of the Customers whose FirstName have 'h' in second position.

```
SELECT FirstName, City, Age FROM Customers WHERE FirstName LIKE '_h%';
```

Output :

```
+-----------+-----------+-----+
| FirstName | City      | Age |
+-----------+-----------+-----+
| Chirag    | Ambarnath |  25 |
| Dhiraj    | Dombivali |  23 |
+-----------+-----------+-----+
```

Is Null Operator :

The IS NULL operator is used to check whether a value in a column is NULL. It returns true if the column value is NULL; otherwise false.

--> Retrieve the FirstName, City and Age from Customers table where the City is null.

```
SELECT FirstName, City, Age FROM Customers WHERE City IS NULL;
```

Output :

```
+-----------+------+-----+
| FirstName | City | Age |
+-----------+------+-----+
| Sham      | NULL |  23 |
+-----------+------+-----+
```

Is Not Null Operator :

The IS NOT NULL operator is used to filter data by verifying whether a particular column has a not-null values.

--> Retrieve the FirstName, City and Age from Customers table where the City is not null.

```
SELECT FirstName, City, Age FROM Customers WHERE City IS NOT NULL;
```

Output :

```
+-----------+-----------+-----+
| FirstName | City      | Age |
+-----------+-----------+-----+
| Saya      | Bhiwandi  |  25 |
| Nikita    | Kalyan    |  23 |
| Parth     | Kalyan    |  22 |
| Chirag    | Ambarnath |  25 |
| Ubaid     | Bhiwandi  |  24 |
| Dhiraj    | Dombivali |  23 |
+-----------+-----------+-----+
```

Built-In-Functions

String Function :

String functions in SQL are used to manipulate and operate on character data stored in columns of a table.

Commonly Used String Functions:

1. CONCAT(str1, str2, ...) :

Concatenates two or more strings together.

--> Concat FirstName and LastName from Customers table.

```
Select CONCAT(FirstName, ' ',  LastName) as FullName from Customers;
```

Output :

```
+------------------+
| FullName         |
+------------------+
| Saya Suryavanshi |
| Nikita Giri      |
| Parth Shirke     |
| Chirag Kamble    |
| Ubaid Fazlani    |
| Dhiraj Anand     |
| Sham Mahajan     |
+------------------+
```

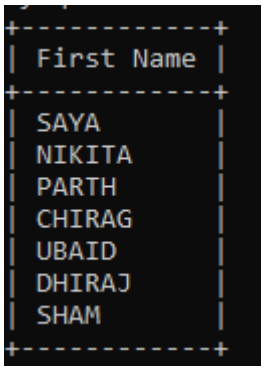
2. UPPER(str):

It Converts a string to uppercase.

--> Convert FirstName in uppercase format from Customers table.

```
Select UPPER(FirstName) as 'First Name' from Customers;
```

Output :



First Name
SAYA
NIKITA
PARTH
CHIRAG
UBAID
DHIRAJ
SHAM

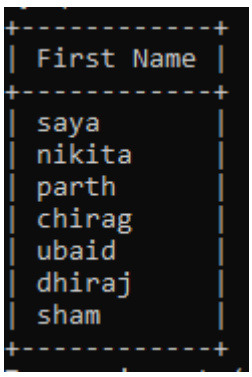
3. LOWER(str):

Converts a string to lowercase.

--> Concat FirstName and LastName from Customers table.

```
Select LOWER(FirstName) as 'First Name' from Customers;
```

Output :



First Name
saya
nikita
parth
chirag
ubaid
dhiraj
sham

4. LENGTH(str):

Returns the length of a string.

--> Find the length of the FirstName from Customers table.

```
Select FirstName, LENGTH(FirstName) as 'Length of Name' from Customers;
```

Output :

FirstName	Length of Name
Saya	4
Nikita	6
Parth	5
Chirag	6
Ubaid	5
Dhiraj	6
Sham	4

5. SUBSTRING(str, start, length):

Extracts a substring from a string.

--> Find the substring of LastName from Customers table.

```
Select LastName, SUBSTRING(LastName,1,5) as 'Substring of Name' from Customers;
```

Output :

LastName	Substring of Name
Suryavanshi	Surya
Giri	Giri
Shirke	Shirk
Kamble	Kambl
Fazlani	Fazla
Anand	Anand
Mahajan	Mahaj

Math Functions :

Math functions in SQL are used to perform mathematical operations on numeric data stored in columns of a table.

Commonly Used Math Functions:

1. ROUND(num, decimal_places):

Rounds a numeric value to a specified number of decimal places.

```
SELECT ROUND(3.14159, 2);
```

Output :

```
+-------------------+
| ROUND(3.14159, 2) |
+-------------------+
|              3.14 |
+-------------------+
```

2. CEIL(num):

Returns the smallest integer greater than or equal to a given number.

```
SELECT CEIL(4.3);
```

Output :

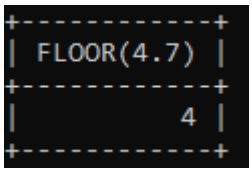
```
+-----------+
| CEIL(4.3) |
+-----------+
|         5 |
+-----------+
```

3. FLOOR(num):

Returns the largest integer less than or equal to a given number.

```
SELECT FLOOR(4.7);
```

Output :

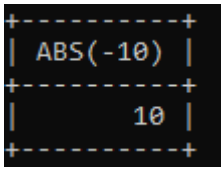


4. ABS(num):

Returns the absolute value of a number.

```
SELECT ABS(-10);
```

Output :

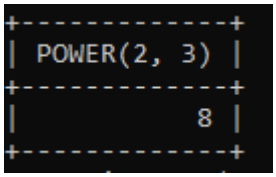


5. POWER(num, exponent):

Returns the value of a number raised to the power of another number.

```
SELECT POWER(2, 3);
```

Output :



Date Function :

Date functions in SQL are used to perform operations on date and time values stored in columns of a table.

Commonly Used Date Functions:

1. CURRENT_DATE:

Returns the current date (without time).

```
SELECT CURRENT_DATE;
```

Output :

```
+--------------+
| CURRENT_DATE |
+--------------+
| 2025-03-03   |
+--------------+
```

2. CURRENT_TIME:

Returns the current time (without date).

```
SELECT CURRENT_TIME;
```

Outout :

```
+--------------+
| CURRENT_TIME |
+--------------+
| 21:04:51     |
+--------------+
```

3. CURRENT_TIMESTAMP:

Returns the current date and time.

```
SELECT CURRENT_TIMESTAMP;
```

Output :

```
+---------------------+
| CURRENT_TIMESTAMP   |
+---------------------+
| 2025-03-03 21:05:18 |
+---------------------+
```

4. DATE_ADD(interval, number, date):

Adds a specified number of intervals (e.g., days, months, years) to a date.

```
SELECT DATE_ADD("2025-01-15", INTERVAL 10 DAY);
```

Output :

```
+----------------------------------------+
| DATE_ADD("2015-01-15", INTERVAL 10 DAY) |
+----------------------------------------+
| 2015-01-25                             |
+----------------------------------------+
```

5. DATEDIFF(interval, date1, date2):

Calculates the difference between two dates in terms of a specified interval (e.g., days, months, years).

```
SELECT DATEDIFF('2025-01-01', '2023-07-09');
```

Output:

```
+-------------------------------------+
| DATEDIFF('2025-01-01', '2023-07-09') |
+-------------------------------------+
|                                 542 |
+-------------------------------------+
```

Aggregate Functions

Aggregate functions in SQL perform a calculation on a set of values and return a single value. They are commonly used with the GROUP BY clause to perform calculations on groups of rows.

Common Aggregate Functions :

1. COUNT():

Counts the number of rows that match a specified condition.

```
SELECT COUNT(*) as 'Number of Medicines' FROM Medicines;
```

Output :

```
+---------------------+
| Number of Medicines |
+---------------------+
|                   7 |
+---------------------+
```

2. SUM():

Calculates the sum of values in a column.

```
SELECT SUM(Price) as 'Total Sum of Price' FROM Medicines;
```

Output :

```
+-----------------+
| Total Sum Price |
+-----------------+
|          379.50 |
+-----------------+
```

3. AVG():

Calculates the average of values in a column.

```
SELECT AVG(Price) as 'Average of Price' FROM Medicines;
```

Output :

```
+------------------+
| Average of Price |
+------------------+
|        54.214286 |
+------------------+
```

4. MIN():

Returns the minimum value in a column.

```
SELECT MIN(Price) as 'Minimum Price' FROM Medicines;
```

Output :

```
+---------------+
| Minimum Price |
+---------------+
|         19.00 |
+---------------+
```

5.MAX():

Returns the maximum value in a column.

```
SELECT MAX(Price) as 'Maximum Price' FROM Medicines;
```

Output:

```
+---------------+
| Maximum Price |
+---------------+
|        110.00 |
+---------------+
```

Subquery

A subquery, also known as a nested query or inner query, is a query nested within another SQL statement (outer query). It allows you to retrieve data from one or more tables based on a condition or criteria defined in the subquery.

The result of the subquery is used by the outer query to further filter or manipulate data.

Guidelines for subquery :

1) Always write subquery within ()

2) Places subquery to right side of operator in query.

3) Do not add order by clause in subquery.

4) Use single row operator in single row query(<, >, <=, >=)

5) Use multiple row operator in multiple row subquery (IN, ALL, Any)

Types of Subqueries :

1. Single-Row Subquery:

Subqueries that return a single row as an output to their parent query are called single-row subqueries. Single-row subqueries are used in a SQL SELECT statement with HAVING clause, WHERE clause, or a FROM clause and a comparison operator.

Single-row subqueries are used in the SELECT statement.

--> Find the Most Expensive Medicine.

```
SELECT Name, Brand, Price, Stock, Description FROM Medicines
WHERE Price = (SELECT MAX(Price) FROM Medicines);
```

Output :

```
+-------------+----------+--------+-------+-----------------------------------------+
| Name        | Brand    | Price  | Stock | Description                             |
+-------------+----------+--------+-------+-----------------------------------------+
| Cough Syrup | Benadryl | 110.00 |    80 | Used for treating cough & throat relief |
+-------------+----------+--------+-------+-----------------------------------------+
```

Explanation:

The subquery finds the highest price from the Medicines table.

The outer query fetches the medicine that matches this price.

2. Multiple-Row-Subquery :

Subqueries that return multiple rows as an output to their parent query are called multiple-row subqueries. Multiple row subqueries can be used in a SQL SELECT statement with a HAVING clause, WHERE clause, a FROM clause, and a logical operator (ALL, IN, NOT IN, and ANY).

A multi-row subquery returns multiple rows (one column) and is used with operators like IN, ANY, ALL, EXISTS, and NOT IN.

--> Find Customers Who Placed Orders.

```
SELECT Cust_id, Name, Phone, Created_at FROM Customers
WHERE Cust_Id IN (SELECT DISTINCT Cust_Id FROM Orders);
```

Output :

```
+---------+---------------+------------+---------------------+
| Cust_id | Name          | Phone      | Created_at          |
+---------+---------------+------------+---------------------+
|     101 | Nikita Giri   | 8765432109 | 2024-09-26 11:30:00 |
|     102 | Parth Shirke  | 7654321098 | 2024-10-15 08:45:00 |
|     103 | Chirag Kamble | 9543210987 | 2024-11-03 09:30:00 |
|     104 | Ubaid Fazlani | 8432109876 | 2024-12-05 10:00:00 |
|     105 | Dhiraj Anand  | 9321098765 | 2025-01-02 11:00:00 |
+---------+---------------+------------+---------------------+
```

Joins

An SQL Join clause is used to combine multiple related tables in a database, based on common fields orcolumns.

1. Inner Join :

The Inner Join is a type of join that combines multiple tables by retrieving records that have matching values in both tables (in the common column).

It compares each row of the first table with each row of the second table, to find all pairs of rows that satisfy the join-predicate. When the join-predicate is satisfied, the column values from both tables are combined into a new table.

Syntax :

```
SELECT column_names....
FROM table1
INNER JOIN table2
ON table1.matching_column = table2.matching_column;
```

Example :

-->Get Orders with User Details.

```
SELECT Orders.Cust_id, Customers.Name, Orders.Total_Price, Orders.Status
FROM Orders INNER JOIN Customers
ON Orders.Cust_id = Customers.Cust_id;
```

Output :

```
+---------+---------------+-------------+-----------+
| Cust_id | Name          | Total_Price | Status    |
+---------+---------------+-------------+-----------+
|     101 | Nikita Giri   |      154.00 | Pending   |
|     102 | Parth Shirke  |       19.00 | Shipped   |
|     103 | Chirag Kamble |      110.00 | Delivered |
|     104 | Ubaid Fazlani |      143.00 | Delivered |
|     105 | Dhiraj Anand  |      182.50 | Pending   |
+---------+---------------+-------------+-----------+
```

2. Left Join

Left Join or Left Outer Join in SQL combines two or more tables, where the first table is returned wholly; but, only the matching record(s) are retrieved from the consequent tables. If zero (0) records are matched in the consequent tables, the join will still return a row in the result, but with NULL in each column from the right table.

Syntax :

```
SELECT column_names....
FROM table1
LEFT JOIN table2
ON table1.matching_column = table2.matching_column;
```

--> Show All Customers and Their Orders

```
SELECT Customers.Name, Orders.Order_ID, Orders.Total_Price, Orders.Status
FROM Customers LEFT JOIN Orders
ON Customers.Cust_id = Orders.Cust_id;
```

Output :

```
+------------------+----------+-------------+-----------+
| Name             | Order_ID | Total_Price | Status    |
+------------------+----------+-------------+-----------+
| Saya Suryavanshi |     NULL |        NULL | NULL      |
| Nikita Giri      |      300 |      154.00 | Pending   |
| Parth Shirke     |      301 |       19.00 | Shipped   |
| Chirag Kamble    |      302 |      110.00 | Delivered |
| Ubaid Fazlani    |      303 |      143.00 | Delivered |
| Dhiraj Anand     |      304 |      182.50 | Pending   |
+------------------+----------+-------------+-----------+
```

left join query, retrieves the details of customers who made an order and who did not. There is no match found for ‘Saya Suryavanshi’, the query return NULL in that record.

3. Right Join :

The Right Join query in SQL returns all rows from the right table, even if there are no matches in the left table. A right join returns all the values from the right table, plus matched values from the left table or NULL in case of no matching join predicate.

Syntax :

```
SELECT column_names....
FROM table1
RIGHT JOIN table2
ON table1.matching_column = table2.matching_column;
```

--> Show All Orders with Customers Info.

```
SELECT Customers.Name, Orders.Order_id, Orders.Total_Price
FROM Customers RIGHT JOIN Orders
ON Customers.Cust_id = Orders.Cust_id;
```

Output :

```
+---------------+----------+-------------+
| Name          | Order_id | Total_Price |
+---------------+----------+-------------+
| Nikita Giri   |      300 |      154.00 |
| Parth Shirke  |      301 |       19.00 |
| Chirag Kamble |      302 |      110.00 |
| Ubaid Fazlani |      303 |      143.00 |
| Dhiraj Anand  |      304 |      182.50 |
+---------------+----------+-------------+
```

4. Full Join :

Full Join creates a new table by joining two tables as a whole. The joined table contains all records from both the tables and fills NULL values for missing matches on either side. Full join combines the result-sets of both left and right joins.

Syntax :

```sql
SELECT column_names....
FROM table1
LEFT JOIN table2
ON table1.matching_column = table2.matching_column

UNION

SELECT column_names....
FROM table1
RIGHT JOIN table2
ON table1.matching_column = table2.matching_column;
```

--> Show All Payments and Orders even if there is no match.

```sql
SELECT Orders.Order_id, Orders.Total_Price, Orders.Status,
Payments.Payment_id, Payments.Payment_Status
FROM Orders Left JOIN Payments
ON Orders.Order_id = Payments.Order_id

Union

SELECT Orders.Order_id, Orders.Total_Price, Orders.Status,
Payments.Payment_id, Payments.Payment_Status
FROM Orders Right JOIN Payments
ON Orders.Order_id = Payments.Order_id;
```

Output :

Order_id	Total_Price	Status	Payment_id	Payment_Status
300	154.00	Pending	NULL	NULL
301	19.00	Shipped	1	Paid
302	110.00	Delivered	2	Paid
303	143.00	Delivered	3	Paid
304	182.50	Pending	4	Pending

5. Cross Join :

Cross Join is a basic type of inner join that is used to retrieve the Cartesian product or cross product of two individual tables. This join will combine each row of the first table with each row of second table.

Syntax :

```
SELECT column_names....
FROM table1
CROSS JOIN table2;
```

--> List All Possible Customers and Orders Combinations

```
SELECT Name, Total_Price, Order_Date
FROM Customers CROSS JOIN Orders;
```

Output :

NAME	Total_Price	Order_Date
Saya Suryavanshi	182.50	2025-03-01 14:20:00
Saya Suryavanshi	143.00	2025-02-16 12:00:00
Saya Suryavanshi	110.00	2025-02-11 09:45:00
Saya Suryavanshi	19.00	2025-01-25 15:00:00
Saya Suryavanshi	154.00	2025-01-02 10:30:00
Nikita Giri	182.50	2025-03-01 14:20:00
Nikita Giri	143.00	2025-02-16 12:00:00
Nikita Giri	110.00	2025-02-11 09:45:00
Nikita Giri	19.00	2025-01-25 15:00:00
Nikita Giri	154.00	2025-01-02 10:30:00
Parth Shirke	182.50	2025-03-01 14:20:00
Parth Shirke	143.00	2025-02-16 12:00:00
Parth Shirke	110.00	2025-02-11 09:45:00
Parth Shirke	19.00	2025-01-25 15:00:00
Parth Shirke	154.00	2025-01-02 10:30:00
Chirag Kamble	182.50	2025-03-01 14:20:00
Chirag Kamble	143.00	2025-02-16 12:00:00
Chirag Kamble	110.00	2025-02-11 09:45:00
Chirag Kamble	19.00	2025-01-25 15:00:00
Chirag Kamble	154.00	2025-01-02 10:30:00
Ubaid Fazlani	182.50	2025-03-01 14:20:00
Ubaid Fazlani	143.00	2025-02-16 12:00:00
Ubaid Fazlani	110.00	2025-02-11 09:45:00
Ubaid Fazlani	19.00	2025-01-25 15:00:00
Ubaid Fazlani	154.00	2025-01-02 10:30:00
Dhiraj Anand	182.50	2025-03-01 14:20:00
Dhiraj Anand	143.00	2025-02-16 12:00:00
Dhiraj Anand	110.00	2025-02-11 09:45:00
Dhiraj Anand	19.00	2025-01-25 15:00:00
Dhiraj Anand	154.00	2025-01-02 10:30:00

6. Self Join :

The SQL Self Join is used to join a table to itself as if the table were two tables. To carry this out, alias of the tables should be used at least once.

Self Join is a type of inner join, which is performed in cases where the comparison between two columns of a same table is required; probably to establish a relationship between them. A table is joined with itself when it contains both Foreign Key and Primary Key in it.

Syntax :

```
SELECT column_names..
FROM table1 a, table1 b
WHERE a.common_field = b.common_field;
```

Alternatively, using the JOIN syntax:

```
SELECT column_names..
FROM table1 a JOIN table1 b
WHERE a.common_field = b.common_field;
```

View

A view is a database object that has no values. Its contents are based on the base table. It contains rows and columns similar to the real table.

The View is a virtual table created by a query by joining one or more tables. It is operated similarly to the base table but does not contain any data of its own.

The View and table have one main difference that the views are definitions built on top of other tables (or views). If any changes occur in the underlying table, the same changes reflected in the View also.

Create View :

Syntax to Create View :

```
CREATE VIEW view_name AS
SELECT columns
FROM tables
[WHERE conditions];
```

Syntax to See View :

```
SELECT * FROM view_name;
```

--> Create a view to Medicines stock less than 100

```
CREATE VIEW ShowAvailableStock_view
AS
SELECT Name, Stock FROM Medicines
where Stock<100;
```

To See view write query as :

```
SELECT view ShowAvailableStock_view ;
```

Output :

```
+-----------------+-------+
| Name            | Stock |
+-----------------+-------+
| Aspirin 75mg    |    90 |
| Cough Syrup     |    80 |
| Vitamin C 500mg |    70 |
| Metformin 500mg |    65 |
+-----------------+-------+
```

Update View :

The ALTER VIEW statement is used to modify or update the already created VIEW without dropping it.

--> Add ExpirayDate and Price in existing 'ShowAvailableStock_view' view.

```
ALTER VIEW ShowAvailableStock_view
AS
SELECT Name, Stock, ExpiryDate, Price
FROM Medicines where Stock<100;
```

To see updates view write a query as :

```
SELECT view ShowAvailableStock_view ;
```

Output :

```
+-----------------+-------+------------+--------+
| Name            | Stock | ExpiryDate | Price  |
+-----------------+-------+------------+--------+
| Aspirin 75mg    |    90 | 2027-01-12 |  25.00 |
| Cough Syrup     |    80 | 2025-07-03 | 110.00 |
| Vitamin C 500mg |    70 | 2027-07-28 |  50.00 |
| Metformin 500mg |    65 | 2028-01-21 |  58.00 |
+-----------------+-------+------------+--------+
```

Drop View :

We can drop the existing view by using the DROP VIEW statement.

```
DROP view ShowAvailableStock_view ;
```

To verify view is available or not write query as :

```
SELECT view ShowAvailableStock_view ;
```

It will give error as :

```
ERROR 1146 (42S02): Table 'onlinemedicinesorderingsystem.showavailablestockex' doesn't exist
```

Store Procedure

Store procedure is a pre-compile collection of sql statements that can be executed as a single unit.

Stored procedures can be used to perform different database operations such as inserting, updating, or deleting data.

Store procedure are store inside the database itself and manage by the database server.

Benefits of Store procedure :

1) Modularity : Store procedure encapsulate sql statement into reusable unit. It promoting code reuse and reducing redundancy.

2) Performance : Executing store procedure can be faster than sending multiple sql queries from a client application to database server.

3) Security : Access to database object can be control through store procedure.

4) Maintenance : Changes made to store procedure are centralize and affect all application that used the procedure reducing maintenance effect.

Basic Syntax to create Store Procedure :

```
DELIMITER //
CREATE PROCEDURE procedure_name
AS
BEGIN
    -- SQL statements
END //
DELIMITER
```

Syntax to Execute Store procedure :

```
Call procedure_name;
```

Example :

--> Create a procedure named 'GetStockOfMedicines' without any parameters to Get Medicines whose Stock is less than 100.

```
CREATE PROCEDURE GetStockOfMedicines()
BEGIN
    SELECT Medicine_id, Name, Stock FROM Medicines WHERE Stock <100;
END //
DELIMITER ;
```

To verify the changes, we execute the procedure using the CALL command as shown in the query below :

```
Call GetStockOfMedicines();
```

Output :

```
+-------------+-----------------+-------+
| Medicine_id | Name            | Stock |
+-------------+-----------------+-------+
|         203 | Aspirin 75mg    |    90 |
|         204 | Cough Syrup     |    80 |
|         205 | Vitamin C 500mg |    70 |
|         206 | Metformin 500mg |    65 |
+-------------+-----------------+-------+
```

Stored Procedure Parameter Types :

Stored procedures can have different types of parameters, which are used to decide the values that will be passed during execution.

1. Input Parameter (IN parameter) :

The IN parameter is the default parameter and is used to receive the input value from the calling program. The value is passed at the time of procedure execution.

-->Create a procedure named ‘GetCustomerOrders ‘ with IN parameter to Get Orders of a Specific Customer.

```
DELIMITER $$
CREATE PROCEDURE GetCustomerOrders(IN CustomerId INT)
BEGIN
    SELECT * FROM Orders WHERE Cust_id = CustomerId;
END $$
DELIMITER ;
```

To execute the SP and pass a value for the 'CustomerId' parameter, we will use the CALL command that takes a customer's ID as an input parameter and returns the Orders details of that Customer.

```
Call GetCustomerOrders(103);
```

Output :

```
+----------+-------------+-----------+---------------------+---------+
| Order_id | Total_price | Status    | Order_date          | Cust_id |
+----------+-------------+-----------+---------------------+---------+
|      302 |      110.00 | Delivered | 2025-02-11 09:45:00 |     103 |
+----------+-------------+-----------+---------------------+---------+
```

2. Output Parameter (OUT parameter) :

The OUT parameter is used to send the output values to the calling program. It is necessary to specify the OUT keyword to an output parameter when creating the procedure. At the time of calling, a variable, prefixed with '@' is used to hold the returned value.

We can then use the SELECT statement on the variable to display the output of the procedure.

--> Create a procedure named 'GetCountOfDeliveredOrders ' with OUT parameter to Get Count of Total Delivered Orders.

```
DELIMITER %%
CREATE PROCEDURE GetCountOfDeliveredOrders(OUT TotalDeliveredOrders INT)
BEGIN
    SELECT COUNT(*) INTO TotalDeliveredOrders FROM Orders WHERE Status = 'Delivered';
END %%
DELIMITER ;
```

The output value will be stored in the '@TotalDeliveredOrders' parameter passed; which is further displayed using the SELECT statement as shown below

```
Call GetCountOfDeliveredOrders(@TotalDeliveredOrders);

Select @TotalDeliveredOrders as 'Count of Delivered Orders';
```

Output :

```
+---------------------------+
| Count of Delivered Orders |
+---------------------------+
|                         2 |
+---------------------------+
```

Cursor

A cursor in mySql is database object that allows you to retrieve result set a row by row rather than all at once.

Cursors are primarily used when we need to process individual rows from a query result.

Perform some operation on each row and move to the next row.

This is particularly used when working with store procedure where you might need to perform complex row by row operations.

How does a cursor work :

1. Declare a cursor :
   This defines the query whose result set will be iterated over.

2. Open the cursor :
   This executes the query and prepare the result set for iteration.

3. Fetch rows from the cursor :
   You can retrieve individual rows from the result set on by one.

4. Process the fetched data :
   You can perform any operations (e.g, updating or inserting data) on the row that was fetched.

5. Close the cursor :
   After all rows are processed, you close the cursor to release system resources.

Steps for using cursor :

- Declare cursor
- Open Cursor
- Loop
- Fetch data from cursor
- Exit loop
- Close cursor

DECLARE statement :

Syntax :

```
DECLARE  cursor_name   CURSOR FOR    select_statement;
```

Explanation :

The cursor declaration must be after any variable declaration.

A cursor must always be associated with a select statement.

Not Found Handler :

- When working with mySql cursor, you must also declare a NOT FOUND handler to handle the situation when the cursor could not find any row.
- Because each time you call the FETCH statement, the cursor attempts to read the next row in the result set.
- When the cursor reaches the end of the result set, it will not be able to get the data, and a condition is raised.
- The handler is used to handler this condition.

Example :

```
DECLARE  CONTINUE  HANDLER  FOR  NOT  FOUND  SET  done=TRUE;
```

Open Cursor :

Syntax : OPEN cursor_name;

Explanation :

The OPEN statement initializes the result set for the cursor, therefore, you must call the OPEN statement before fetching rows from the result set.

LOOP Statement :

Syntax :

[label_name :] LOOP

```
Statement_list
```

END LOOP[label_name]

Explanation :

The LOOP depends on the careful placement of the LEAVE statement to terminate iteration.

Fetch Rows from the Cursor:

Syntax :

FETCH cursor_name INTO variable list;

Explanation :

Then, you use the FETCH statement to retrieve the next row pointed by the cursor and move the cursor next row in the result set.

Close cursor :

CLOSE cursor_name;

Close the cursor to release resources.

Example :

```
--> Write a Cursor to Find Medicines Expiring Soon
DELIMITER  $$
CREATE PROCEDURE  GetExpiringMedicines()
BEGIN
   DECLARE done INT DEFAULT 0;
   DECLARE med_name VARCHAR(100);
   DECLARE exp_date DATE;

   -- Declare a cursor to fetch medicines expiring in the next 3 months
   DECLARE  expiring_cursor CURSOR FOR
```

```
    SELECT name, ExpiryDate FROM medicines WHERE ExpiryDate <= DATE_ADD(CURDATE(), INTERVAL 6 MONTH);


    -- Declare handler for loop termination
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;


    -- Open the cursor
    OPEN expiring_cursor;


    -- Loop through all matching rows
    read_loop: LOOP
        FETCH expiring_cursor INTO med_name, exp_date;
        IF done = 1 THEN
            LEAVE read_loop;
        END IF;


        -- Print the medicine details
        SELECT CONCAT('Medicine: ', med_name, ' is expiring on ', exp_date) AS 'Expiring Medicine';
    END LOOP;


    -- Close the cursor
    CLOSE expiring_cursor;
END $$
DELIMITER ;
```

--> To execute Cursor :

```
Call GetExpiringMedicines();
```

Output :

```
+-----------------------------------------------------+
| Expiring Medicine                                   |
+-----------------------------------------------------+
| Medicine: Paracetamol 500mg is expiring on 2025-06-30 |
+-----------------------------------------------------+
```

```
+-------------------------------------------------+
| Expiring Medicine                               |
+-------------------------------------------------+
| Medicine: Cough Syrup is expiring on 2025-07-03 |
+-------------------------------------------------+
```

Trigger

Trigger is defined as a response to an event. In MySQL, a trigger is a special stored procedure that resides in the system catalogue, that is executed automatically (without being called explicitly like regular stored procedures) whenever an event is performed.

These events include statements like INSERT, UPDATE and DELETE etc.

Triggers are usually divided into two categories −

Row-level Trigger: Triggers that are only executed when each row is either inserted, updated or deleted in a database table. MySQL only supports these type of triggers.

Statement-level Trigger: Triggers like these are executed on the transaction level, once, no matter how many rows are modified in a table. MySQL does not support these type of triggers.

Types of Triggers

There are six types of row-level triggers in MySQL. They are:

1. Before Insert Trigger

2. After Insert Trigger

3. Before Update Triggerssss

4. After Update Trigger

5. Before Delete Trigger

6. After Delete Trigger

Advantages of Triggers :

- Triggers help the database to maintain the integrity of the data stored.
- Triggers are also a means to handle errors from the database layer itself.
- As triggers are invoked automatically without being called explicitly, you don't have to wait for the scheduled events to run.

- Triggers can be useful to track the data changes made in the tables, by logging the events.
- MySQL Triggers can also prevent invalid transactions from being executed.

How to write triggers :

Syntax :

```
Delimiter $$
create trigger trigger_name
before/after    inert|update|delete
on table_name
for each row
begin
        statements
end $$
delimiter ;
```

Example :

--> Write a Trigger to update stock when order is placed.

```
DELIMITER //
CREATE TRIGGER UpdateStockAfterOrder
AFTER INSERT ON Order_Items
FOR EACH ROW
BEGIN
   UPDATE Medicines
   SET Stock = Stock - NEW.Quantity
```

```
    WHERE Medicine_id = NEW.Medicine_id;
END //
DELIMITER ;
```

To check Trigger write query :

Select * from Medicines;

Select * from Order_Items;